

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในปริญญานิพนธ์

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการวิจัย และพื้นฐานของตัวภาษาที่ใช้ในระบบของโปรแกรมตัวนี้

2.1 ภาษา XML (Extensible Markup Language)

คอมโพเนนต์พื้นฐานของ XML คือ เอเลเมนต์ (Element) , แอตทริบิวต์ (Attribute) และคอมเมนต์ (Comment)

2.1.1. เอเลเมนต์ (Element)

เอเลเมนต์ (Element) เป็นสิ่งที่นำมาใช้เพื่อกำหนดเครื่องหมาย บางส่วนของเอกสาร XML ซึ่ง XML Element มีรูปแบบตามตัวอย่าง 2.1 ต่อไปนี้

`<ElementName>Content</ElementName>` (ตัวอย่าง 2.1)

เนื้อหาจะอยู่ภายในแท็ก XML โดย XML ครอบเนื้อหาภายในไว้ แต่สามารถมีเอเลเมนต์ที่ไม่มีเนื้อหาอยู่ภายในก็ได้ เราเรียกว่า Empty Element ใน XML สามารถเขียนได้ตามตัวอย่าง 2.2 ดังนี้

`<ElementName/>` หรือ `<ElementName></ElementName>` (ตัวอย่าง 2.2)

เอเลเมนต์ซ้อนเอเลเมนต์ สามารถซ้อนกันได้ตัวอย่างเช่นถ้าเราต้องการรวมกลุ่มข้อมูลทั้งหมดไว้ภายใต้เอเลเมนต์ชื่อเดียวกันทำได้ดังนี้

```
<Patient>
  <PatientName>Name</PatientName>
  <PatientAge>Age</PatientAge>
  <PatientWeight>Weight</PatientWeight>
</Patient>
```

(ตัวอย่าง 2.3)

ดังนั้น XML Element สามารถบรรจุเอเลเมนต์อื่น ๆ ได้ อย่างไรก็ดีตาม แท็กเริ่มต้นทุกแท็กต้องมีแท็กปิดท้ายที่สอดคล้องกันเสมอ

ชื่อของเอเลเมนต์ต้องตั้งตามกฎต่อไปนี้

- ชื่อต้องไม่ประกอบด้วยอักขระช่องว่าง ไม่ว่าจะเป็นเพียงหนึ่งช่องว่างหรือมากกว่า ถ้าชื่อเป็นอักขระตัวเดียว อักขระนั้นต้องเป็นพยัญชนะที่สามารถเป็นได้ทั้งตัวพิมพ์ใหญ่ (A-Z) หรือตัวพิมพ์เล็ก (a-z)
- ชื่อสามารถเริ่มต้นด้วยพยัญชนะหรือเครื่องหมาย Underscore (_)
- นอกจากอักขระตัวแรกเราสามารถใช้อักขระตัวใดก็ได้
- ชื่อของเอเลเมนต์มีลักษณะเป็น Case Sensitive (คือการคำนึงถึงตัวอักษรใหญ่หรือเล็กด้วย) ดังนั้น PatientName , PATIENTNAME และ patientname เป็นเอเลเมนต์ที่แตกต่างกัน

ตัวอย่างที่ 2.4 ชื่อของเอเลเมนต์ต่อไปนี้เป็นรูปแบบที่ถูกต้อง

Fred

_Fred

(ตัวอย่าง 2.4)

Fredd123

FredGrüß

ตัวอย่างที่ 2.5 ชื่อของเอเลเมนต์ต่อไปนี้เป็นรูปแบบที่ไม่ถูกต้อง

Fred 123

-Fred

(ตัวอย่าง 2.5)

123

จากตัวอย่าง 2.5 ชื่อเอเลเมนต์แรกประกอบไปด้วยช่องว่างระหว่างชื่อ ชื่อที่สองเริ่มต้นด้วยขีด (Dash) และชื่อที่สามเริ่มต้นด้วยตัวเลขแทนที่จะเป็นพยัญชนะหรือเครื่องหมาย Underscore (_)

2.1.2. แอตทริบิวต์ (Attribute)

แอตทริบิวต์ (Attribute) เป็นกลไกสำหรับการเพิ่มรายละเอียดข้อมูลไปยังเอเลเมนต์ เช่น ในเรคอร์ด PatientWeight ของเอกสาร XML เราไม่ทราบว่า PatientWeight มีหน่วยของน้ำหนักเป็นปอนด์หรือกิโลกรัม เพื่อให้ทราบว่า PatientWeight มีหน่วยน้ำหนักเป็นปอนด์ เราจะต้องเพิ่มแอตทริบิวต์ Unit และระบุค่าเป็น LB ตามตัวอย่างที่ 2.6

< PatientWeight Unit="LB" > Weight < /PatientWeight > (ตัวอย่าง 2.6)

แอตทริบิวต์สามารถรวมอยู่ในเฉพาะแท็กเริ่มต้นเท่านั้นและมีลักษณะที่เหมือนอิลิเมนต์
คือเป็น Case Sensitive ค่าของแอตทริบิวต์ต้องถูกครอบอยู่ในเครื่องหมาย Double Quotation
(" ") แอตทริบิวต์สามารถใช้ร่วมกับ Empty Element เช่น ตามตัวอย่าง 2.7 ต่อไปนี้

< PatientWeight Unit="LB" / > (ตัวอย่าง 2.7)

เราสามารถประกาศแอตทริบิวต์ได้เพียงครั้งเดียวเท่านั้นในอิลิเมนต์

2.1.3. คอมเมนต์ (Comment)

คอมเมนต์ (Comment) คอมเมนต์เป็นรายละเอียดที่ฝังอยู่ในเอกสาร XML เพื่อให้ข้อมูล
เพิ่มเติมที่อยู่ภายในเอกสารคอมเมนต์ใน XML ในรูปประโยค (Syntax) ที่เหมือนกับรูปแบบ
คำสั่งที่ใช้ใน HTML ซึ่งทำให้เข้าไปในขณะที่แอปพลิเคชันประมวลผลเอกสาร ตามตัวอย่าง
2.8

<!-- Comment text --> (ตัวอย่าง 2.8)

คำสั่งในการประมวลผล Processing Instruction (PI) ต้องเริ่มต้นด้วย "<?" และตาม
ด้วยชื่อของ Application ที่จะรับข้อมูล PItarget และข้อความที่จะส่งผ่านไปยัง application ตาม
ตัวอย่าง 2.9

<?nameprocessor SELECT * FROM blah?> (ตัวอย่าง 2.9)

ซึ่งคำสั่งนี้จะถูกส่งไปยัง Parsers แต่จะไม่ถูกส่งไปยัง Application

2.1.4. XLink, XPath , Xpointer

XLink, XPath , Xpointer ความสามารถในการเชื่อมโยงเอกสารหนึ่งไปยังอีกเอกสาร
หนึ่ง

2.1.4.1 XLink คือการเชื่อมโยงไปเอกสารหนึ่งทั้งเอกสาร

Attribute ของ XLink ที่นำมาใช้กับ Element เป็นดังนี้

- type ใช้กำหนดชนิดของ element ของ XLink ที่กำลังจะสร้าง เช่น

xlink:type = "simple" ใช้สำหรับ element แบบ simple

- href คือ attribute บอกตำแหน่งใช้กำหนด URI(Uniform Resource Identifier)ในการดึงข้อมูลเช่น

<myelementxlink:herf="http://sernaferna.com/home.htm">

- role ซึ่งถูกใช้ใน link ส่วนขยายเพื่อกำหนดหน้าที่ของแหล่งข้อมูล link ในรูปแบบที่เครื่องสามารถเข้าใจได้ ใช้แสดงความหมายและนัยสำคัญของลิงค์ให้กับแอปพลิเคชันซอฟต์แวร์จุดประสงค์เพื่อจัดเตรียมข้อมูลโดยรวมกับลิงค์ ซึ่งมนุษย์ไม่สามารถอ่านทำความเข้าใจได้ การใช้งานแอตทริบิวต์ ROLE ทำให้เราจัดเตรียม Application ร่วมกับข้อมูลรายละเอียดเกี่ยวกับ link มากกว่าที่จะเรียนรู้แต่เพียงว่า link ไปที่ใด

- title ซึ่งถูกใช้ใน link ส่วนขยายเพื่อกำหนดหน้าที่ของแหล่งข้อมูล link ในรูปแบบที่มนุษย์สามารถเข้าใจได้ title จะถูกนำไปสร้าง tool tip ซึ่งจะปรากฏขึ้นถ้าผู้ใช้เลื่อนเมาส์ไปวางอยู่เหนือลิงค์

- show กำหนดวิธีการที่แหล่งข้อมูลจะถูกนำมาแสดงผล เมื่อถูกอ่านเข้ามา ซึ่งมีค่าที่เป็นไปได้อยู่ 4 ค่าคือ

1. new ใช้ระบุว่าแหล่งข้อมูลจะถูกแสดงผลในหน้าต่างแยกต่างหาก
2. replace ใช้ระบุว่าเอกสารปัจจุบันถูกแทนที่ด้วยเอกสารใหม่
3. embed ใช้ระบุว่า xlink element นั้นจะถูกแทนที่ด้วยแหล่งข้อมูลใด เช่น รูปภาพ
4. undefined แอปพลิเคชันจะมีอิสระในการเชื่อมโยงกับลิงค์ด้วยวิธีที่เห็นว่าเหมาะสม

- actuate ใช้กำหนดว่าเมื่อไรที่แหล่งข้อมูลที่กำหนดจะถูกอ่านเข้ามาใช้ ซึ่งมีค่าที่เป็นไปได้อยู่ 3 ค่าคือ

1. onLoad แหล่งข้อมูลจะถูกดึงเข้ามาทันทีที่เอกสารนั้นถูกโหลด
2. onRequest แหล่งข้อมูลจะถูกดึงก็ต่อเมื่อผู้ใช้ต้องการมัน
3. undefined แอปพลิเคชันจะมีอิสระในการเชื่อมโยงกับลิงค์ด้วยวิธีที่เห็นว่าเหมาะสม

- Attribute from และ to เพื่อกำหนดทิศทางของ link โดยที่ link ของ xlink นั้นไม่จำเป็นต้องเป็นแบบทิศทางเดียว

ชนิดของ link

- Simple link เป็นลิงค์แบบทิศทางเดียวที่เชื่อมโยงแหล่งข้อมูลเพียงสองแหล่ง

- Extended link คือสิ่งที่เชื่อมความสัมพันธ์ระหว่างแหล่งข้อมูลจำนวนใดจำนวนหนึ่งเข้าด้วยกัน

- Locator ถูกใช้สำหรับชี้แหล่งข้อมูลหนึ่งที่อยู่ห่างไกลที่เป็นส่วนหนึ่งของ link
- Arc ถูกใช้สำหรับชี้การท่องจากแหล่งข้อมูลหนึ่งไปยังอีกแหล่งข้อมูลหนึ่งใน link
- Resource ใช้กำหนดแหล่งข้อมูลหลากหลายที่เป็นส่วนหนึ่งของ link
- Title สามารถให้อ่านข้อมูล title ของ link ที่มนุษย์อ่านเข้าใจได้

2.1.4.2 XPath

XPath คือภาษาที่ใช้เพื่อหาและได้ส่วนที่ต้องการใน XML files ส่วนที่ต้องการนั้นระบุโดยใช้ path ซึ่งคล้ายกับ path ที่ใช้ในการหา file ในคอมพิวเตอร์

XPath ทำหน้าที่ในการชี้ตำแหน่ง (address) ของส่วนต่าง ๆ ของเอกสาร XML เพื่อที่จะอนุญาตให้เราได้รับชิ้นข้อมูลที่เรากำลังต้องการได้

การสร้าง Path บอกตำแหน่ง

จะเริ่มค้นจากรากของเอกสาร (root node) เสมออย่างเช่น “/order/item/part-number” หมายถึง ให้เลือก element ใด ๆ ที่มีชื่อว่า <part-number> ซึ่งเป็นลูกของ element <item> ซึ่งเป็นลูกของ element <order> ซึ่งเป็นรากของเอกสารนั่นเอง

2.1.4.3 XPointer

XPointer ได้จัดเตรียมการเข้าถึงค่าของแอตทริบิวต์หรือเนื้อหาของเอลเมนต์ใดที่อยู่ในเอกสาร XML ตำแหน่งใดก็ตาม การใช้ XPointer ทำคุณสามารถคลิกไปยังส่วนต่างๆ ของข้อความ หรือเลือกที่จะไปคู่แอตทริบิวต์หรือเอลเมนต์ที่ต้องการ และไปยังเอลเมนต์ต่างๆ เป็นต้น ข้อกำหนดลักษณะลักษณะเฉพาะไม่ได้เป็นตัวกำหนดว่าแอปพลิเคชันใดสามารถทำงานร่วมกับข้อมูลที่ถูกล็อกด้วย XPointer

2.1.5 โอเปอเรเตอร์ (Operator)

การกำหนดตำแหน่งของโหนดนั้นขึ้นอยู่กับชื่อของมันโดยไม่สนใจว่ามันมาจากส่วนไหนในเอกสารโดยใช้สัญลักษณ์ “//” เช่น “//customer” จะเลือกเอา element <customer> ใด ๆ ที่มีอยู่ในเอกสารมา โดยไม่สนใจว่ามันขึ้นอยู่กับ element ไหน

XPointer

การเชื่อมโยงไปอีกเอกสารหนึ่งเฉพาะส่วนที่เราต้องการ (นี่คือข้อแตกต่างจาก xlink) เช่น

[http://www.sernaferna.com/default/page.xml#xpointer\(//section-one\)](http://www.sernaferna.com/default/page.xml#xpointer(//section-one))

จะเห็นว่าเมื่อใส่ xpointer เข้าไปแอปพลิเคชันที่รับรู้ xpointer จะสนใจในส่วน of element <section-one> เท่านั้น และจะเห็นว่ามันคล้าย ๆ กับ xpath ซึ่งแท้จริงแล้ว xpointer นั้นสร้างขึ้นมาจาก xpath แต่จะไม่อ้างถึงโหนดอีกต่อไป แต่จะใช้คำว่าตัวบอกตำแหน่ง (location) แทน ซึ่งตัวบอกตำแหน่งเหล่านี้สามารถเป็นได้ทั้งโหนดที่ใช้ใน xpath และโหนดชนิดใหม่ คือ

point หรือ range ตัวใดตัวหนึ่งก็ได้ นอกจากนี้แทนที่จะสัมพันธ์กับโหนดเซต มันจะสัมพันธ์กับ location set แทน

2.1.6 Namespaces

Namespaces หมายถึงอะไรก็ตามที่เราใช้จำแนก element ของเอกสาร XML และในบางครั้งเรายังใช้จำแนก attribute ได้อีกด้วยเนื่องจากโปรแกรมเมอร์อาจจะใส่ข้อมูลที่แตกต่างกันลงไป element เดียวกัน เช่นในกรณีที่เราต้องผสมผสาน XML ที่เป็นของเราเองและ XHTML element ที่หลากหลายจากเอกสารหลายประเภทเข้าไปในเอกสาร XML ฉบับเดียว

XML Namespaces ทำงานอย่างไร

การที่จะใช้ XML Namespaces ในเอกสารของเรานั้น element ทั้งหมดก็ยังจำเป็นต้องถูกกำหนดค่าด้วยชื่อที่ตรงตามคุณสมบัติที่กำหนด หรือ QName ซึ่งจะประกอบไปด้วยส่วนประกอบสองส่วนนั่นคือ ส่วนเฉพาะ (local par) คือชื่อทั้งหลายที่เรากำหนดให้กับ element มาโดยตลอดและ Namespaces Prefix ซึ่งระบุว่าชื่อดังกล่าวนี้ขึ้นอยู่กับ Namespaces ไດ

2.1.7 DTD และ XML Schema

การกำหนดโครงสร้างของเอกสาร XML ทำได้ 2 วิธีหลัก ๆ คือ DTD และ XML Schema โดยไม่ว่าจะกำหนดตามวิธีใดก็แล้วแต่ เรียกว่าเอกสาร XML นั้น “valid”

เหตุผลที่ต้องกำหนดโครงสร้างของเอกสาร ก็เพื่อให้เอกสาร XML นั้นมีรูปแบบที่ถูกต้องพร้อมที่จะนำไปใช้งาน เพราะธรรมชาติของเอกสาร XML ก็คือการแลกเปลี่ยนข้อมูลระหว่างองค์กรหรือหน่วยงาน ดังนั้นฝ่ายที่รับเอกสาร XML จากองค์กรอื่น ต้องมั่นใจว่าเอกสารนั้นมีความถูกต้อง สามารถนำไปใช้งานได้ พุดให้ง่าย ๆ อีกอย่างหนึ่งก็คือ DTD หรือ XML Schema เปรียบเหมือน “สัญญา” ระหว่างหน่วยงาน หรือผู้ที่เกี่ยวข้องกับเอกสาร XML นั้น ๆ เมื่อใดก็ตามที่มีการสร้างเอกสาร XML ขึ้นมาเก็บข้อมูล จะต้องปฏิบัติตามข้อกำหนดใน DTD หรือ XML Schema หากไม่ทำตาม ก็ถือว่าผิดสัญญาและเอกสาร XML นั้นย่อมนำไปใช้งานไม่ได้

สรุปความสำคัญของ DTD และ XML Schema ได้เป็น 2 แ่งมุมคือ

1. เป็นการกำหนดรูปแบบโครงสร้างของเอกสาร XML (หรือเรียกว่า Data Model) ว่าประกอบไปด้วยแท็กอะไรบ้าง , มี Attribute อะไรบ้าง , ค่าของ Attribute เป็นอะไรได้บ้างหรือลักษณะการซ้อนแท็กเป็นอย่างไรได้บ้างเป็นต้น
2. เป็นสัญญาระหว่างองค์กร ที่จะแลกเปลี่ยนข้อมูลกันในรูปแบบของ XML เพื่อให้เอกสารที่สร้างขึ้นมามีรูปแบบตรงตามที่ตกลงกันไว้ และจะได้สามารถนำเอกสาร XML นั้นไปใช้งานได้ถูกต้อง

2.1.8 XSL

เอกสาร XSL คล้ายกับเอกสาร CSS ในกรณีที่ทั้งสองเอกสารมีการกำหนดสไตล์ที่ประยุกต์ไปยังเอเลเมนต์ที่แน่นอน แต่มีสิ่งที่แตกต่างกันเล็กน้อย CSS กำหนดสไตล์การพิมพ์

เฉพาะ XHTML Element เท่านั้น ในทางตรงกันข้าม สไลด์ที่กำหนดเอกสาร XSL จะประยุกต์ไปยังเอกสาร XML เอกสาร XSL ทั้งเอกสารอาจใช้สไลด์ที่กำหนดใน CSS เพื่อสร้างโค้ดจากข้อมูล XML เอกสาร XSL ต้องวางบนเซิร์ฟเวอร์เดียวกัน รวมทั้งไฟล์ที่อ้างถึงเอกสารดังกล่าวด้วย

2.1.9 XSLT

เป็นข้อเสนอแนะจาก W3C ในการกำหนดชุด Element ของ XML ที่สามารถจะถูกใช้ในการสร้างสไลด์ซีทีทีแปลงเอกสาร XML ให้อยู่ในรูปแบบใด ๆ ซึ่งในปัจจุบันนี้ใช้ในการแปลง XML รูปหนึ่งไปเป็นอีกรูปแบบหนึ่ง โดยการทำงานจะทำโดยนำเอกสาร XML ที่เหมาะสมกับ XML ไปทำการแปลงรูปแบบเพื่อสร้างเอกสารเอาต์พุต ซึ่งขึ้นอยู่กับคำสั่งและเนื้อหาภายใน สไลด์ซีทีทีจึงสามารถสร้างรูปแบบเอาต์พุตใด ๆ ก็ได้ตามที่ต้องการในการทำงาน

2.1.10 XML Element

XSLT ใช้โอเลเมนต์ที่เราใช้กับ XSL ในลักษณะที่เหมือนกัน โอเลเมนต์ต่อไปนี้ซึ่งไม่มีการเปลี่ยนแปลง ได้แก่ apply-templates , attribute , element , comment , for-each , copy , if , choose , when และ otherwise โดย pi Element ที่ใช้ใน XSL จะเรียกว่า processing instruction ใน XSLT

XSLT โอเลเมนต์ต่อไปนี้ใน XSL นั้นไม่ได้มีอยู่ใน XSLT ได้แก่ script , cdata , entity-ref และ eval Element

2.1.11 การแปลง XSL ไปเป็น XSLT

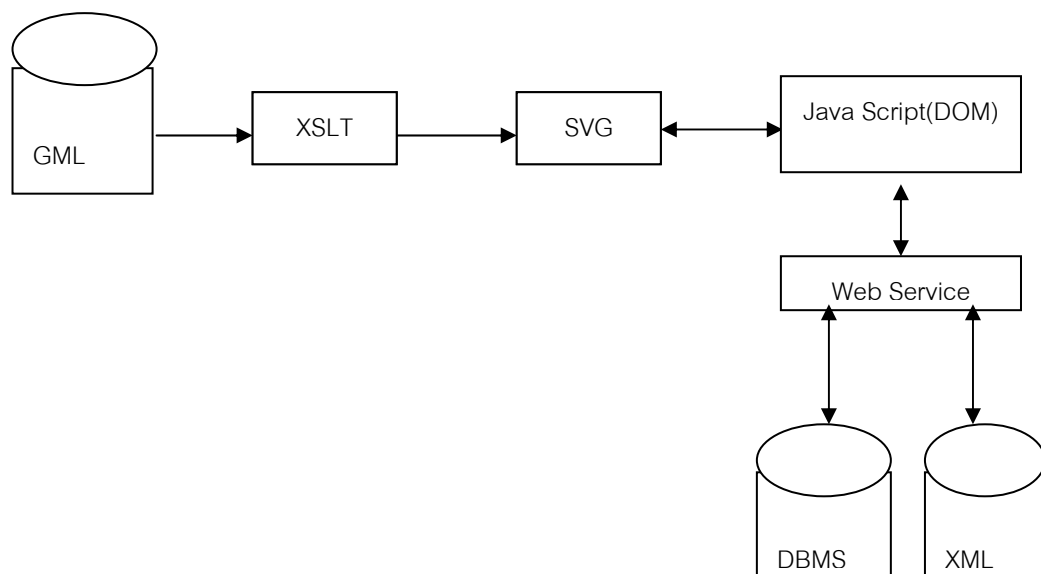
เราจำเป็นต้องอัปเดต XSL Style Sheet เพื่อให้เป็นไปตามข้อกำหนดของ W3C โดยเฉพาะอย่างยิ่ง เราจำเป็นต้องเปลี่ยนแปลง XSL และ Namespace ที่ส่งออกมา เนื่องจากบล็อก <xsl:eval> และ <xsl:script> ไม่ได้รับการสนับสนุนใน XSLT แต่จะมีการเปลี่ยนแปลงไปยังกลไกของ XSLT เรายังต้องเพิ่ม Jscript ของฟังก์ชัน XSL Runtime ในที่ๆจำเป็น เนื่องจากอ็อบเจกต์ XSL Runtime เป็นส่วนเพิ่มเติมอื่นๆ คือ เรายังต้องมีการใส่เครื่องหมายทั้งหมดที่เทียบเท่า XML อย่างเช่น \$ITS\$ จะถูกแทนที่ด้วยเครื่องหมาย □IT , \$gt\$ จะถูกวางด้วยเครื่องหมาย □gt และอื่นๆ เครื่องหมายที่ไม่เท่ากับ XML ดังเช่น ieq นั้น ต้องเขียนฟังก์ชันพิเศษในการรองรับเอง

2.2 ภาษา GML (Geography Markup Language)

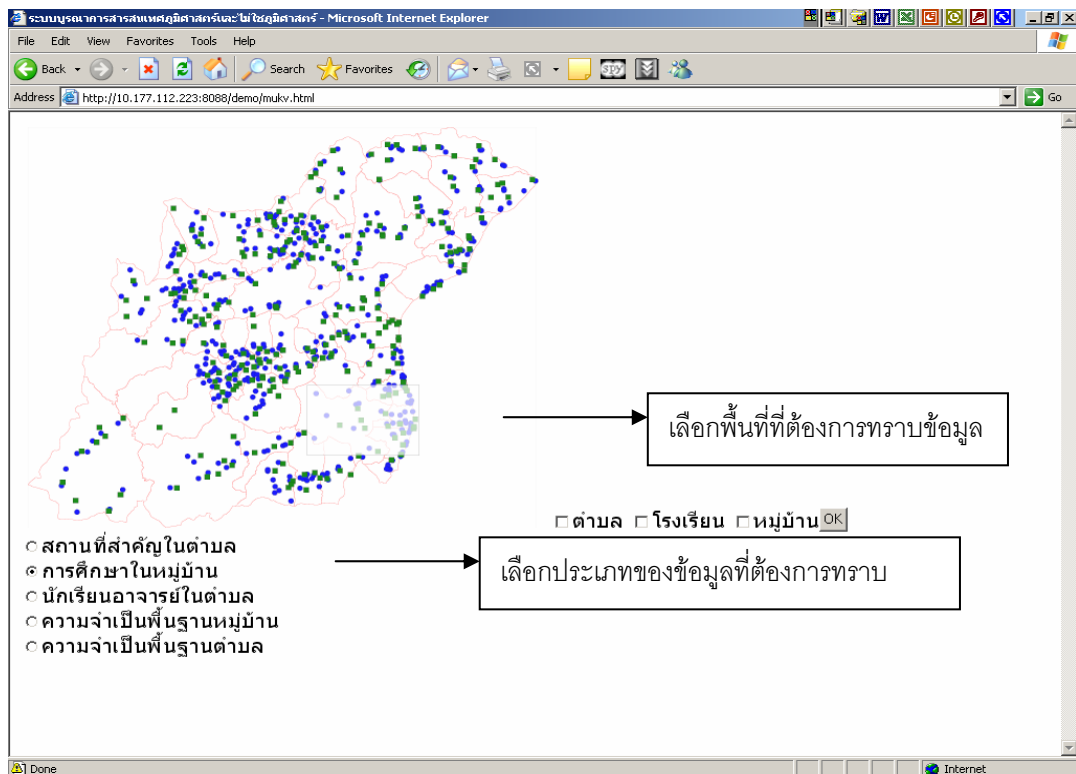
ภาษา GML (Geography Markup Language) เปรียบเสมือนที่เก็บข้อมูลภูมิศาสตร์ในรูปแบบของ XML มาแล้วแปลงข้อมูลด้วยภาษา XSLT ให้อยู่ในรูปแบบของ SVG เพื่อแสดงผลเป็นรูปภาพของแผนที่ และสามารถที่จะโต้ตอบกับผู้ใช้ได้โดยแทรก Code ของ JavaScript เข้าไปเมื่อเกิดการเลือกอาณาบริเวณในแผนที่ที่จะใช้ Java Script ที่แทรกอยู่ใน SVG File เพื่อใช้ javascript DOM ค้นหาข้อมูลใน GML ที่เป็นข้อมูลที่มีความสัมพันธ์กันกับพื้นที่ที่เลือกนั้นแล้วนำข้อมูลที่ได้จากการค้นหานั้นโดยข้อมูลที่ได้อาจเป็น ข้อมูลในส่วนที่ใช้ในการติดต่อกับ

Webservice ว่าใช้ Webmethod ไหน หรือใช้ข้อมูลในตารางไหน หรือจะ parameter อะไรไป เพื่อที่จะให้ Webservice แล้ว Java Script ก็จะติดต่อกับ Webservice โดยใช้ Soap Protocol ทางฝั่ง ของ Webservice ก็จะนำข้อมูลที่รับจาก Client มาประมวลผลเพื่อหาข้อมูลที่ต้องการและส่ง ผลลัพธ์กลับไปเพื่อแสดงผลในฝั่ง Client

หลักการในการดำเนินงานได้แบ่งเป็น 2 ส่วน คือ ส่วนภูมิสารสนเทศ กับส่วนสารสนเทศ ในส่วนภูมิสารสนเทศ ได้ใช้เทคโนโลยี ของ GML มาเป็นส่วนเก็บข้อมูลพิกัดของแผนที่และ แสดงผลแผนที่ด้วย SVG ในส่วนของ สารสนเทศได้ใช้ Program C#.net มาพัฒนาในส่วนของ Webservice และใช้ Microsoft Access เก็บข้อมูลสารสนเทศ และใช้ Java Script เปรียบเสมือน ตัวกลางระหว่างข้อมูลทั้งสองคือเป็นตัวรับข้อมูลจากผู้ใช้ว่าต้องการทราบข้อมูลในส่วนไหนและ นำข้อมูลนั้นไปติดต่อกับ Web service และรับข้อมูลกลับมาเพื่อแสดงผล



รูปที่ 2.1 แสดงภาพรวมขั้นตอนการทำงาน

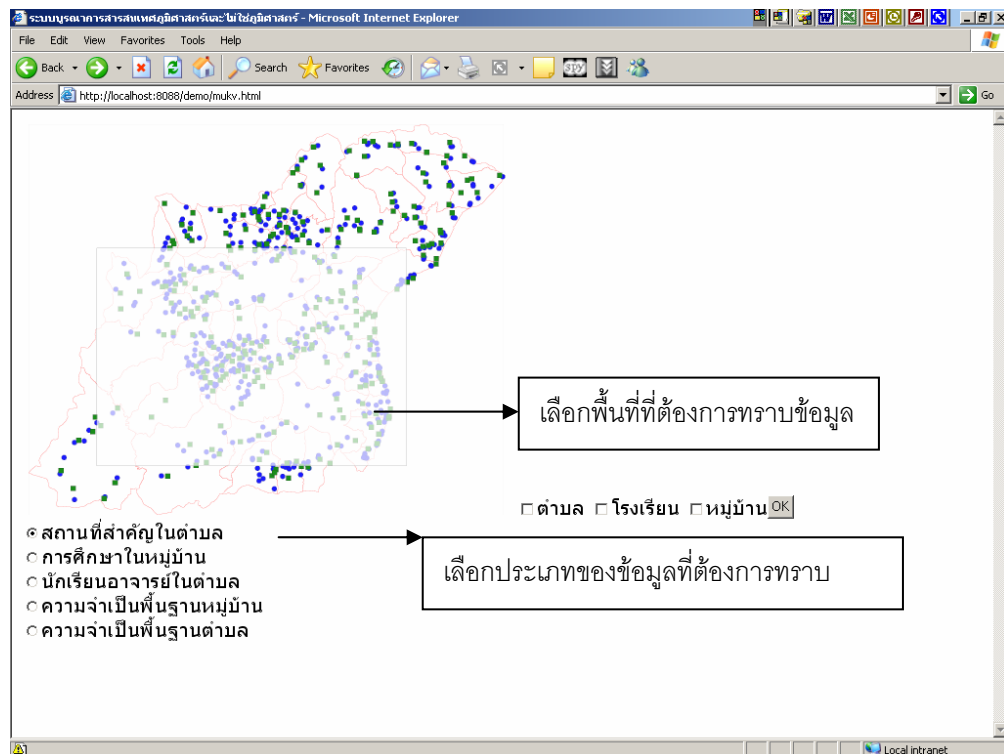


รูปที่ 2.2 Program ที่ได้ทดลอง Implement ในระดับชั้นหมู่บ้าน

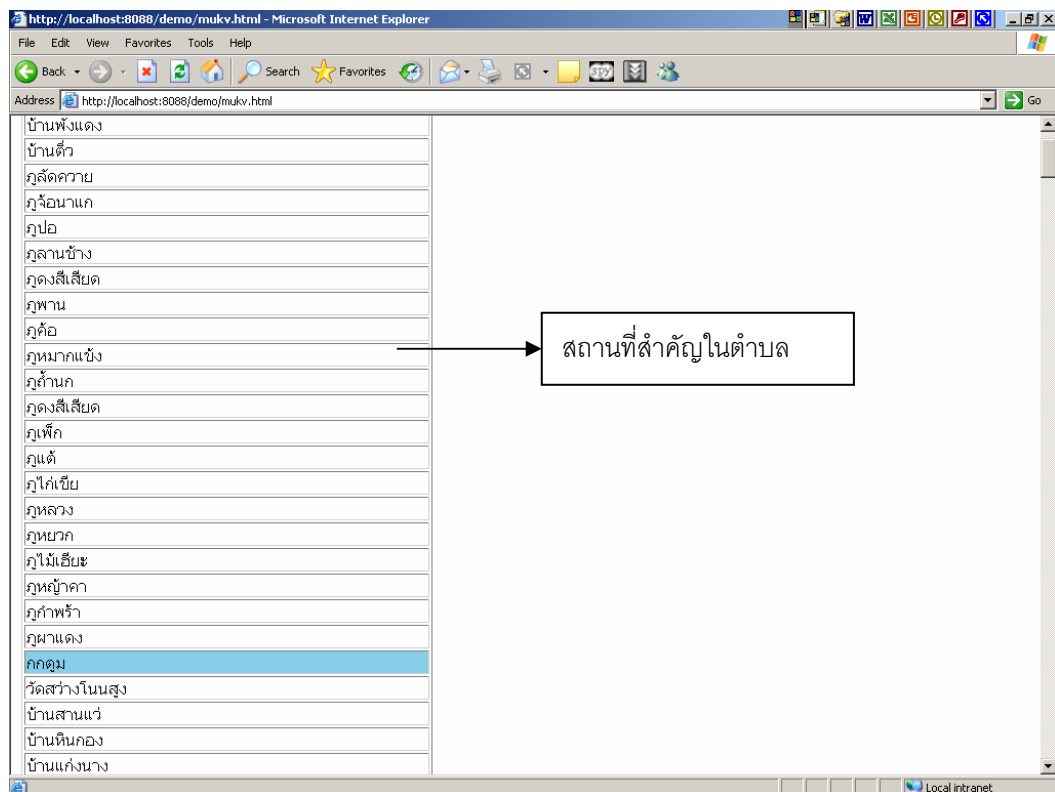
หมู่บ้าน	ผู้จบการศึกษาภาคบังคับ	ประชากรทั้งหมด	ร้อยละ
กุดโง้งใหญ่	515	791	65
คันคำ	77	282	27
โนนสว่าง	303	411	74
หนองแอก	438	800	55
หนองหอย	866	1258	69
ป่าหวาย	211	422	50
ดอนม่วย	322	494	65
โคกสูง	269	835	32
บางทรายใหญ่	537	826	65
บางทรายใหญ่	703	1017	69
สงเปลือย	140	238	59
คำฝักนอก	117	201	58
ดอนม่วงพัฒนา	130	167	78
คำฮี	911	1158	79
นาโสกน้อย	287	360	80
แก่นเต่า	486	673	72
หนองหญ้าไซย	687	893	77
โพนทราย	449	646	70
โพนทราย	160	399	40
ม่วงหัก	656	776	85
ดงยาง	492	698	70
บ้านเนินหัน	246	330	75
สมสะอาด	359	443	81
กุดแซ่	798	950	84

แสดงผลการศึกษาของหมู่บ้าน

รูปที่ 2.3 แสดงรายละเอียดของข้อมูลที่ถูกเลือก



รูปที่ 2.4 แสดงการเลือกพื้นที่และรายละเอียด



รูปที่ 2.5 แสดงรายละเอียดข้อมูลที่ถูกเลือก

ข้อดีในการเลือกใช้ภาษา GML ในการแสดงผลที่

- ได้แผนที่ที่มีคุณภาพดีขึ้น เนื่องจาก GML จะเปลี่ยนข้อมูลทางภูมิศาสตร์ทั้งหมดให้อยู่ในรูปของ feature หรือ object ซึ่งสามารถจะนำไปแสดงผลในรูปแบบที่ไม่จำกัดความละเอียดในการแสดงผล
- สามารถใช้งานได้บนบราวเซอร์โดยไม่ต้องซื้อโปรแกรมที่ใช้แสดงผลทางฝั่ง Client เมื่อข้อมูล GML ถูกรับโดยฝั่ง Client จะถูกเปลี่ยนให้อยู่ในรูปของภาพแผนที่บนบราวเซอร์โดยใช้ภาษา SVG ในการแสดงออกมาเป็นภาพ ซึ่งเราสามารถแสดงแผนที่บนบราวเซอร์ โดยไม่จำเป็นต้องติดตั้งซอฟต์แวร์อื่น ๆ เพิ่มเติมบราวเซอร์สามารถรองรับภาพแบบเวกเตอร์ได้ และสามารถดาวน์โหลด SVG ซึ่งเป็นโปรแกรมเสริมของ Adobe Corporation ได้โดยไม่ต้องเสียค่าใช้จ่ายใด ๆ
- สามารถสร้างแผนที่ที่มีรูปแบบเฉพาะได้ เนื่องจาก GML จะเก็บข้อมูลของแผนที่ เช่น ที่ตั้งของ feature นั้น , ลักษณะทางภูมิศาสตร์ , ชนิดของ feature และ attribute ต่าง ๆ โดยไม่กำหนดข้อมูลหรือวิธีในการแสดงของข้อมูล แผนที่นั้น ๆ ซึ่งทำให้ผู้ใช้สามารถสร้างแผนที่ให้มีรูปแบบได้ตามที่ต้องการ
- สามารถทำการแก้ไขแผนที่ได้ แผนที่ที่เขียนโดย GML จะถูกดาวน์โหลดและทำการ render บนบราวเซอร์ซึ่งเมื่อ GML ถูกเปลี่ยนเป็น SVG ผู้ใช้จะสามารถใช้โปรแกรมบน Client ในการเพิ่มเติมตัวอักษร หรือทำการเปลี่ยนแปลงภาพบนแผนที่ และสามารถบันทึกเก็บเป็นไฟล์ภาพเพื่อนำไปใช้ในกรณีอื่น เช่น การพิมพ์ออกทางเครื่องพิมพ์หรือทำการส่ง E-mail ได้
- สามารถเชื่อมโยง link ไปลงในแผนที่ได้ ประโยชน์ที่สำคัญของ GML คือผู้ใช้สามารถฝัง URL ลงใน feature ที่มีแผนที่ ความสามารถนี้ทำให้สามารถออกแบบแผนที่ให้มีความแปลกใหม่และทันสมัยยิ่งขึ้น
- ทำการ Query ได้ดียิ่งขึ้น หากผู้ใช้งานต้องการที่จะ click ไปบน feature บนแผนที่และค้นหาข้อมูลเพิ่มเติมเกี่ยวกับ feature นั้น ๆ เช่น ชื่อหรือความลึกของแม่น้ำ ซึ่งไม่สามารถทำได้บนแผนที่ที่สร้างขึ้นโดยใช้ไฟล์ภาพ (GIF / JPG) สำหรับแผนที่ที่ใช้ GML นั้นเราจะสามารถระบุ feature ที่ถูก click นั้นได้
- สามารถกำหนดเนื้อหาที่ต้องการแสดงหรือต้องการได้รับเพราะ GML เป็นภาษาที่อ้างอิงถึง feature ทำให้ผู้ใช้สามารถกำหนดได้ว่าต้องการให้มี feature ใดแสดงบนแผนที่บ้าง ซึ่งจะช่วยลดระยะเวลาในการรับ

- สามารถแสดงผลได้บนทุกอุปกรณ์ที่ใช้เบราว์เซอร์ได้ ข้อมูลทางภูมิศาสตร์ใน GML สามารถส่งไปยังอุปกรณ์ชนิดใดก็ได้ที่สามารถรองรับ XML การแสดงผลแบบ XML ได้เช่น เครื่อง PDA หรือโทรศัพท์มือถือทำให้แผนที่นั้นสามารถใช้งานได้อย่างกว้างขวาง

2.3 ภาษา SVG (Scalable Vector Graphics)

2.3.1 ความรู้เบื้องต้นเกี่ยวกับภาษา SVG

SVG มี XML เป็นพื้นฐาน ที่สร้างขึ้นเพื่อกำหนดลักษณะการแสดงผลในรูปแบบสองมิติโดย SVG จะกำหนดรูปแบบของวัตถุสามอย่างคือ vector graphic shape (เช่น ส่วนของเส้นตรง และเส้นโค้ง), รูปและตัวอักษร โดยสามารถที่จะกำหนด grouped, styled, transformed หรือ composited ให้แก่วัตถุใน SVG ได้ อีกทั้งยังเพิ่มคุณสมบัติ Transformations, clipping paths, alpha masks, filter effects, template object และอีกมากมาย เนื่องจาก SVG เป็น XML ดังนั้นจึงมีการสร้าง DOM สำหรับ SVG โดยอ้างอิงตามมาตรฐานของ W3 ซึ่งทำให้ SVG มีคุณสมบัติ dynamic และ interactive ตามไปด้วยแม้แต่ event handlers อย่าง onmouseover หรือ onclick ก็สามารถกำหนดลงในแต่ละวัตถุของ SVG ได้ด้วยอีกทั้งการอ้างอิงตามมาตรฐานทำให้ SVG สามารถคุยกับ XML อื่น ๆ ใน web page เดียวกันได้

SVG มีสองส่วนคือส่วนที่เป็น XML-based file format และส่วน programming API สำหรับ graphical application SVG สร้างขึ้นบนมาตรฐานอื่น ๆ มากมายเช่น XML JPEG PNG DOM SMIL CSS เป็นต้น ความสามารถในการขยาย SVG และ embed metadata มีประโยชน์ในการ mapping community ตัวอย่างเช่น graphical element สามารถระบุ native object เช่นทะเลสาบได้ ยอมให้ application interact กับ object ในลักษณะ graphical

SVG เป็นส่วนประกอบที่สมบูรณ์แบบสำหรับ OpenGIS ที่เป็น GML format GML ก็เป็น XML-based ที่ใช้อธิบาย geographical element เช่น แม่น้ำและถนน GML สามารถ convert เป็น SVG โดยใช้ XML pipeline สำหรับ online display

2.3.2 Link ของ SVG

SVG มี 'a' element คล้ายกับ 'a' element ของ HTML ที่ใช้ระบุ link ซึ่ง SVG ใช้ XLink สำหรับทุก link definitions

SVG 1.0 ต้องการแค่ให้ user agents support XLink's notion ของ simple links โดยแต่ละ simple link เกี่ยวข้องกับเฉพาะ 2 resources เท่านั้นคือ 1 local และ 1 remote ด้วย arc ที่ชี้จาก former ไปยัง latter

Simple link หนึ่งนั้นกำหนดสำหรับหนึ่ง render element เท่านั้นดังนั้นถ้า ‘a’ element มี 3 ‘circle’ element จะต้องมีการสร้าง link สำหรับแต่ละ circle และแต่ละ rendered element ใน ‘a’ element คือ local resource (source anchor สำหรับ link)

Remote resource (ปลายทางของ link) จะกำหนดโดย URI ซึ่งระบุโดย XLink href attribute บน ‘a’ element โดย remote resource นั้นอาจเป็น Web resource ใด ๆ ก็ได้ เช่น ภาพ , video clip , sound bite , program , SVG document อื่น , HTML document , element ใน document ปัจจุบัน , element ใน document อื่น เป็นต้น) เมื่อทำการ activate link เหล่านี้ user จะไป visit resource เหล่านั้น



รูปที่ 2.6 ภาพแผนที่ ในรูปแบบ SVG ที่แสดงบน Web Brower

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 20001102//EN"
3 "http://www.w3.org/TR/2000/CR-SVG-20001102/DTD/svg-20001102.dtd">
4 <svg width="678" height="400" viewBox="672356 -1556851.93 2705 1771" onzoom="zooming0">
5 <desc>Thammasat Internet Map</desc>
6 <defs>
7 <script xlink:href="/data/Thammasat_Internet_Map.js" type="text/javascript">
8 </script>
9 </defs>
10
11 <g id="1">
12 <g id="AsiagameResident" style="fill:#FFC7AE;stroke:#000000;stroke-width:0.50">
```

รูปที่ 2.7 แสดง tag ของ SVG

2.4 การติดต่อกับฐานข้อมูล

2.4.1 PostgreSQL

เป็น Object-Relational DBMS โดยสามารถใช้รูปแบบของภาษา SQL ได้เกือบทั้งหมด และสามารถใช้ subselects , transactions , user-defined types และ functions ได้ อีกทั้งเป็น Database ซึ่งให้ Source code ฟรี ด้วย

ประวัติการพัฒนา

- เป็น Project ของ Prof. Michael Stonebraker ที่ มหาวิทยาลัย Berkeley
- ซึ่งเดิมพัฒนามาจาก Ingres ซึ่งเป็นที่รู้จักในชื่อ CA-Ingres II ซึ่ง Ingres ใช้ ภาษา query , QUEL เป็นภาษาของตัวเอง ปัจจุบันได้หยุดพัฒนาไปแล้ว แต่ยังสามารถนำมาใช้ได้ฟรี
- ต่อมา Prof. Stonebraker ได้นำมาพัฒนาเป็น Postgres (มาจาก after Ingres) ซึ่งได้ใช้ ภาษา query เป็น POSTQUEL เป็น Postgres version 4.2
- ต่อมาในช่วง ปี 1987 Postgres ได้เสนอ rules , procedures , time travel , extensible types และ object-relational concepts
- Postgres ถูกนำมาใช้ เพื่อการค้า ในชื่อว่า Illustra (ปัจจุบัน ถูก Informix ซื้อไป และรวมเข้าไว้ใน Universal Server)
- ต่อมา นักศึกษาปริญญาเอก 2 คน คือ Andrew Yu และ Jolly Chen ได้พัฒนา Postgres ให้ใช้ภาษา query ตามรูปแบบที่เป็นมาตรฐาน แทนที่ ภาษา POSTQUEL เดิม ซึ่งได้เผยแพร่ในปี 1995 จึงเรียกเป็น Postgres95 หรือ version 5 หลังจากนั้น การพัฒนาต่อ โดย กลุ่มพัฒนาทาง Internet
- ปัจจุบัน เปลี่ยนชื่อเป็น PostgreSQL พัฒนาต่อเนื่องเป็น version 7.1

ลักษณะโครงสร้าง

ระบบที่ใช้ PostgreSQL จะติดตั้ง PostgreSQL ไว้ที่เครื่อง Server ซึ่งเป็นที่เก็บ database ด้วย และยังสามารถ ติดตั้ง PostgreSQL ได้มากกว่า 1 ชุดใน Server เครื่องเดียว

ผู้ดูแลระบบ PostgreSQL จะใช้ชื่อว่า postgres ซึ่งเป็นผู้ดูแลทั้ง ตัวโปรแกรม และ database ซึ่งสามารถทำงานกับบางคำสั่งเฉพาะ เพื่อจัดการ database และ ผู้ใช้บริการ (user) ซึ่งผู้ดูแลระบบ database (postgres) จะคล้ายการทำงานของ superuser ในระบบ Unix หน้าที่ของ postgres สามารถ สร้างชื่อ user และกำหนดสิทธิและระดับการใช้งานต่างๆได้

PostgreSQL ใช้รูปแบบการทำงาน แบบ Client/Server ซึ่งในการทำงานจะประกอบด้วย 3 process ทำงานร่วมกัน คือ

1. Postmaster เป็น supervisory daemon process ซึ่งจัดการติดต่อระหว่าง Frontend กับ Backend process ในการ allocate share buffer , จัดการค่าเริ่มต้นต่างๆในระหว่างเริ่มทำงาน และเก็บบันทึกการเข้าใช้ระบบและความผิดพลาดต่างๆที่เกิดขึ้น

2. Postgres เป็น backend process เพื่อจัดการ database ถือว่าส่วนนี้เป็น process ที่ทำงานจริงๆ เช่น ทำงานตาม query โดย Postmaster จะสั่งให้สร้าง Backend process สำหรับทุกๆ การเชื่อมต่อกับ Frontend ดังนั้น Postgres นี้จะทำงานที่ server

3. Frontend เป็น application ซึ่งจะทำงานที่เครื่อง client และจะส่งคำสั่งการเชื่อมต่อหรือคำสั่งต่างๆ มาที่ Postmaster แล้ว Postmaster จึงส่งต่อการทำงานไปที่ Postgres

2.4.1.1 หลักการทำงานของ PostgreSQL

การทำงานจะแบ่ง process ที่ทำงานดังที่กล่าวมาแล้ว คือ

1. ในส่วนของ Supervisory daemon process คือ Postmaster
2. ในส่วนของ User's Frontend application เช่น โปรแกรม psql หรือ CGI-Perl

3. และในส่วน Backend database servers คือ Postgres

เมื่อโปรแกรม ทาง Frontend ต้องการข้อมูล หรือทำงานกับ database โดยเรียกผ่านทาง library libpq ซึ่ง library libpq นี้ จะส่ง requests ผ่านทาง Network ไปยัง Postmaster เมื่อ Postmaster ได้รับ request ดังกล่าว ทาง Postmaster จะสร้าง Backend process ขึ้นที่ server เพื่อติดต่อกับ Frontend แทน การทำงานนั้นจะเกิดขึ้นระหว่าง Frontend กับ Backend โดยไม่ผ่าน Postmaster อีก และ Postmaster ก็ทำงานต่อไป คือรอรับ request อื่นๆต่อไป

Library libpq จะให้ หนึ่ง Frontend สามารถติดต่อกับได้หลาย Backend processes แต่การทำงานยังเป็นแบบ single threaded เนื่องจาก library libpq ยังไม่สามารถทำ multithreaded ได้

ตามหลักการที่กล่าวมา ดังนั้น Postmaster กับ Backend จะต้องทำงานอยู่ที่เครื่องเดียวกัน คือ database server แต่ Frontend จะทำงานที่เครื่องใดก็ได้

2.4.1.2 ภาษา query ที่ใช้

ภาษา query ที่ PostgreSQL ใช้อิงตามมาตรฐาน SQL3 ซึ่งกำลังจะเป็น มาตรฐานของภาษา Query โดยมีการเพิ่มคุณสมบัติต่างๆ เช่น extensible type system, inheritance, functions และ production rules จากภาษา POSTQUEL เดิม และยังเป็นไปตามมาตรฐาน ANSI SQL ซึ่งประกอบด้วยคำสั่ง คือ

abort transaction, alter table, begin transaction, change acl, close, copy, create aggregate, create database, create function, create index, create operator, create rule, create table, create type, create version, create view, declare, delete, drop aggregate, drop database, drop function, drop index, drop operator, drop rule, drop table, drop type, drop view, end transaction, extend index, fetch, grant, insert, load, purge, revoke, select, update, vacuum

expressions สามารถเป็นได้ทั้ง arbitrary arithmetic, logical , functional expressions , aggregates.

2.4.2 PostGIS

PostGIS เป็นการเพิ่มเติมในส่วน of ฐานข้อมูลเชิงวัตถุสัมพันธ์ (Object-relational database system) ของ PostgreSQL ให้มีการรองรับวัตถุทางระบบสารสนเทศภูมิศาสตร์ (GIS object) เข้ามาเก็บไว้ในฐานข้อมูล มีการสนับสนุน GiST indexes และ R-tree indexes ซึ่งเป็นวิธีการค้นข้อมูลแบบตัวชี้ (Indexing) ที่ใช้ในฐานข้อมูลเชิงพื้นที่สำหรับฐานข้อมูลที่มีขนาดใหญ่ ซึ่ง PostGIS เองมีการกำหนดการใช้งานโดย OpenGIS ที่เป็นลักษณะพื้นฐานของ SQL (SFSQL)

2.4.2.1 ลักษณะทั่วไปของ PostGIS

ดังที่กล่าวมาแล้วว่า PostGIS มีการกำหนดการใช้งานที่เป็นลักษณะพื้นฐานโดย OpenGIS Consortium (OGC) ซึ่งเป็นสถาบันที่ศึกษาเพื่อการสร้างอินเทอร์เน็ตที่ทำงานโปรแกรมประยุกต์ซอฟต์แวร์ ให้มีการใช้งานได้กับระบบสารสนเทศภูมิศาสตร์ (GIS)

OpenGIS เป็นสิ่งกำหนดความชัดเจนในการเข้าถึงข้อมูลทางด้านธรณีที่แตกต่างกัน และประมวลผลทางธรณีของแหล่งทรัพยากรในสภาพแวดล้อมที่เป็นเครือข่าย (Network)

ซึ่งลักษณะที่มีการกำหนดโดย OpenGIS มีดังนี้

- OpenGIS SFSQL Objects เป็นการกำหนดในส่วน of วัตถุเชิงพื้นที่ เช่น
 - POINT
 - LINESTRING
 - POLYGON
 - MULTIPOINT
 - MULTILINESTRING
 - MULTIPOLYGON
 - GEOMETRYCOLLECTION

- OpenGIS SFSQL Representations เป็นการกำหนดในส่วน of การแสดงผล ซึ่งเป็นมาตรฐานในการแสดงผลของวัตถุเชิงพื้นที่ มี 2 รูปแบบคือ

1). Well – Known Text (WKT) Form

เป็นการแสดงผลในรูปแบบที่เป็น String เช่น

POINT(1 1)

MULTIPOINT(1 1, 3 4, -1 3)

LINESTRING(1 1, 2 2, 3 4)

POLYGON((0 0, 0 1, 1 1, 1 0, 0 0))

MULTIPOLYGON((0 0, 0 1, 1 1, 1 0, 0 0), (5 5, 5 6, 6 6, 6 5, 5 5))

MULTILINESTRING((1 1, 2 2, 3 4),(2 2, 3 3, 4 5))

2). Well – Known Binary (WKB) Form

เป็นการแสดงผลในรูปแบบที่เกี่ยวกับบิต ซึ่งจะดึงข้อมูลออกจากฐานข้อมูลโดยไม่มีการ
เปลี่ยนไปแสดงในรูปแบบที่เป็น String

2.4.2.2 การใช้รูปแบบมาตรฐาน OpenGIS

OpenGIS เป็นการระบุลักษณะพื้นฐานสำหรับ SQL ซึ่งจะกำหนดมาตรฐาน
ระบบสารสนเทศภูมิศาสตร์ในส่วนที่เป็นชนิดของวัตถุ ฟังก์ชันที่ใช้ในการจัดการ และกลุ่มของ
ตาราง

ตารางของ OpenGIS ที่ใช้มี 2 ตาราง ดังนี้

1) The SPATIAL_REF_SYS Table มีรูปแบบในการกำหนดดังต่อไปนี้

```
CREATE TABLE SPATIAL_REF_SYS (
```

```
    SRID          INTEGER NOT NULL PRIMARY KEY,
```

```
    AUTH_NAME     VARCHAR(256),
```

```
    AUTH_SRID     INTEGER,
```

```
    SRTEXT        VARCHAR(2048),
```

```
    PROJ4TEXT     VARCHAR(2048)
```

```
)
```

SRID

ตัวเลขที่ใช้ในการระบุระบบการอ้างอิงเชิงพื้นที่ภายในฐานข้อมูล

AUTH_NAME

ชื่อมาตรฐาน หรือ ส่วนที่เป็นมาตรฐาน สำหรับระบบการอ้างอิง

AUTH_SRID

หมายเลขของระบบการอ้างอิงเชิงพื้นที่ ซึ่งมีการกำหนดโดยการอ้างอิง

จาก AUTH_NAME

SRTEXT

เป็นการแสดงผลแบบ Well – known Text ของระบบการอ้างอิงเชิง

พื้นที่

PROJ4TEXT

เป็น Proj4 library ซึ่งใช้กำหนด String ให้กับ SRID

2) The GEOMETRY_COLUMNS Table มีรูปแบบในการกำหนดดังต่อไปนี้

```
CREATE TABLE GEOMETRY_COLUMNS (  
    F_TABLE_CATALOG    VARCHAR(256) NOT NULL,  
    F_TABLE_SCHEMA     VARCHAR(256) NOT NULL,  
    F_TABLE_NAME       VARCHAR(256) NOT NULL,  
    F_GEOMETRY_COLUMN  VARCHAR(256) NOT NULL,  
    COORD_DIMENSION    INTEGER NOT NULL,  
    SRID               INTEGER NOT NULL,  
    TYPE               VARCHAR(30) NOT NULL  
)
```

F_TABLE_CATALOG, F_TABLE_SCHEMA, F_TABLE_NAME

ลักษณะพิเศษของชื่อตารางที่ประกอบด้วยคอลัมน์เชิงเรขาคณิต

F_GEOMETRY_COLUMN

ชื่อของคอลัมน์ทางเรขาคณิต ซึ่งมีลักษณะเป็นตาราง

COORD_DIMENSION

มิติเชิงพื้นที่ของคอลัมน์

SRID

หมายเลขของระบบอ้างอิงเชิงพื้นที่ ที่ใช้สำหรับเรขาคณิตในตาราง ซึ่ง
เป็นคีย์นอกที่อ้างอิงถึง SPATIAL_REF_SYS

TYPE

ชนิดของวัตถุเชิงพื้นที่

2.4.2.3 การสร้าง Spatial table

การนำข้อมูล spatial มาสร้างลง table นั้นมีวิธีทำ 2 ขั้นตอน

1). สร้าง table ธรรมดาของ non-spatial ตัวอย่างเช่น

```
CREATE TABLE ROADS_GEOM ( ID int4, NAME varchar(25) )
```

2). เพิ่ม column ใน table ของ spatial โดยใช้ function OpenGIS
“AddGeometryColumn” ซึ่งจะมี syntax ดังตัวอย่างต่อไปนี้

```
AddGeometryColumn(<schema_name>, <table_name>,  
                  <column_name>, <srid>, <type>,  
                  <dimension>)
```

Or, using current schema:

```
AddGeometryColumn(<table_name>,  
                  <column_name>, <srid>, <type>,  
                  <dimension>)
```

2.5 ShapeFile

2.5.1 ทำไมต้องเป็น Shapefiles

Shapefile จะมีการเก็บแบบเรขาคณิตไม่คงที่ (Nontopological geometry) และ สำหรับ ลักษณะจุดเด่นในชุดข้อมูล ลักษณะเด่นของการเก็บลักษณะของข้อมูลจะเก็บเป็นแบบชุดของ ทิศทางพิกัด (Vector coordinates) เพราะ Shapefile จะไม่มีการประมวลผลส่วนเกินที่จำเป็นของ โครงสร้างข้อมูลรูปแบบเรขาคณิต ข้อดีของ Shapefile จะมีความสามารถตรงที่ความเร็วในการ วาดและการแก้ไขจะทำให้เร็วกว่าข้อมูลอื่น ๆ จุดเด่นอีกอย่างของ Shapefile จะสามารถ ซ้อนทับกัน หรือสามารถแยกเป็นชั้นได้ และง่ายต่อการอ่านและเขียนบน disk ที่มีพื้นที่น้อย

Shapefiles สามารถรองรับ จุด , เส้น และพื้นที่ลักษณะพิเศษต่าง ๆ ได้ โดยพื้นที่พิเศษนี้ อาจจะเป็นแบบ Polygon , Polyline ก็ได้ Attribute ถือว่าอยู่ในรูปแบบของ dBASE ซึ่งแต่ละ Attribute จะมีความสัมพันธ์เป็นแบบ หนึ่งต่อหนึ่ง เกี่ยวข้องกับการกำหนดรูปแบบที่บันทึก เอาไว้

สามารถสร้าง Shapefile ได้ 4 วิธีด้วยกันดังนี้

1. การนำออก (Export) Shapefile สามารถสร้างโดยการนำข้อมูลจาก source มาทำเป็น Shapefile โดยใช้ ARC/INFO , PC ARC/INFO , Spatial Database Engine (SDE) , ArcView GIS หรือ BusinessMAP software
2. การเปลี่ยนให้เป็นตัวเลขโดยใช้เครื่องคอมพิวเตอร์ (Digitize) ซึ่ง Shapefiles สามารถสร้างได้โดยตรงโดยการเปลี่ยนรูปร่างให้เป็นตัวเลข โดยใช้ tool ในการ สร้างของArcView GIS
3. การเขียนโปรแกรม (Programming) โดยใช้ Avenue™ (ArcView GIS) , MapObjects™ , ARC Macro Language (AML™) (ARC/INFO) , หรือ Simple Macro Language (SML™) (PC ARC/INFO) software เราสามารถสร้าง Shapefile ภายในโปรแกรมเหล่านี้ได้
4. การเขียนโปรแกรมให้สร้าง Shapefile ได้โดยตรง

2.5.2 Shapefile Technical Description

ESRI shapefile จะประกอบไปด้วย main file , index file และตาราง dBASE โดย main file จะเข้าถึงโดยตรง variable-record-length file ในแต่ละ record จะบรรยายถึงรูปร่างร่วมกับรูปทรงเรขาคณิตต่าง ๆ ส่วน index file ในแต่ละ record จะประกอบไปด้วย offset ที่คล้ายกันกับ record ของ mainfile จากส่วนแรกของ main file ส่วนตาราง dBASE จะประกอบด้วย attribute ร่วมกับหนึ่ง record ต่อ feature ความสัมพันธ์แบบ one-to-one ระหว่าง geometry และ attribute เป็นแบบ base on record number โดย attribute record ใน dBASE file จะคล้ายกันกับ record ของ main file

2.5.3 Naming Convention

ชื่อทุก file ให้การสนับสนุน 8.3 naming convention โดย main file , index file และ dBASE file มีส่วนที่เติมหน้าคำเหมือนกัน ซึ่งส่วนที่เติมหน้าคำจำเป็นอย่างยิ่งร่วมกับที่ใช้ตัวหนังสือและตัวเลขเป็นสัญลักษณ์ (a-Z , 0-9) ติดตามโดยศูนย์ หรือมากกว่า 7 ตัวอักษร (a-Z , 0-9 , _ , -) ส่วนต่อท้ายคำสำหรับ main file จะเป็น .shp ส่วนต่อท้ายคำของ index file จะเป็น .shx สำหรับส่วนต่อท้ายคำของตาราง dBASE จะเป็น .dbf ชื่อของ file ทุก ๆ ตัวจะเป็นไปตามระบบปฏิบัติการ

<i>Examples</i>	Main file:	counties.shp
	Index file:	counties.shx
	dBASE table:	counties.dbf

2.5.4 Numeric Type

เกี่ยวกับชนิดของตัวเลข Shapefile จะเก็บเป็นแบบ integer และ double-precision โดยมันจะจำชนิดตามแบบเอกสารที่มันอ้างอิงดังนี้

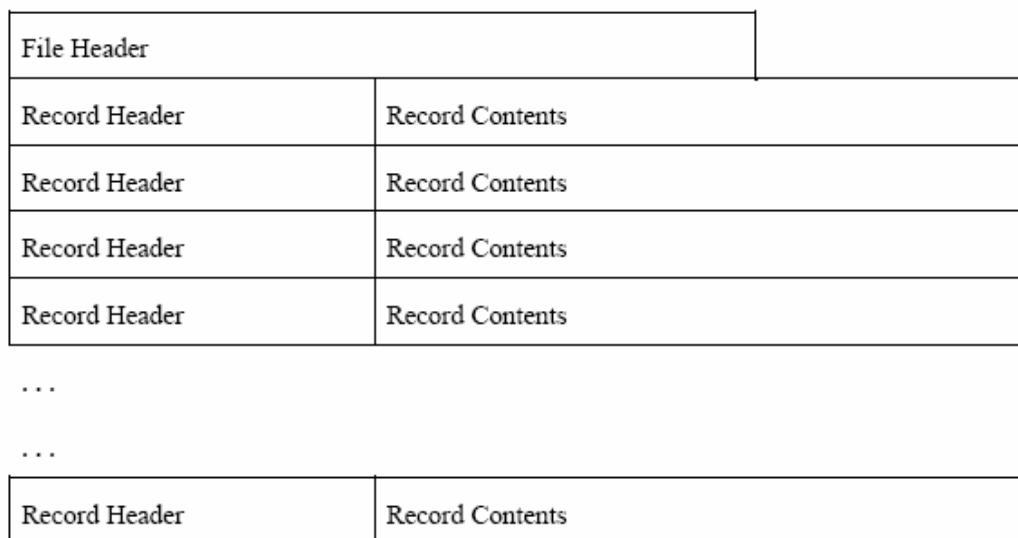
Integer : Signed 32-bit integer (4 byte)

Double : Signed 64-bit IEEE double-precision floating point number (8 byte)

การแทนตัวเลขด้วยจุดทศนิยม (floating point) ทั้งฝั่งบวก , ฝั่งลบ และที่ไม่เป็นตัวเลขใน Shapefile จะไม่ยินยอม อย่างไรก็ตาม Shapefile จะรองรับแนวความคิดของ “no data” แต่ในปัจจุบันจะใช้สำหรับเปรียบเทียบเพียงอย่างเดียว ทุกการแทนตัวเลขด้วยจุดทศนิยม (floating point) จะน้อยกว่า -10^{38} เป็นการไต่ตรงโดย Shapefile reader แทนค่าของ “no data”

2.5.5 องค์ประกอบของ Main File

ส่วนประกอบ main file จะจำกัดความยาว (fixed-length) file ส่วนหัวโดยการบันทึก variable-length record แต่ละ variable-length record เป็นการสร้างขึ้นของ fixed-length record header ตามส่วนประกอบของ variable-length record ในรูปที่ 2.8 เป็นการแสดงองค์ประกอบของ main file



รูปที่ 2.8 องค์ประกอบของ main file

2.5.5.1 Byte Order

ทุกส่วนประกอบใน Shapefile สามารถแบ่งออกได้เป็น 2 ส่วนประกอบ ดังนี้

1. ข้อมูลที่สัมพันธ์กัน (Data related)
 - Main file record contents
 - Main file header's data description fields (Shape Type, Bounding Box, etc.)
2. การจัดการความสัมพันธ์ file (File management related)
 - File and record lengths
 - Record offsets, and so on

integers และ double-precision integers นั้นแทนการบรรยาย field ใน file ส่วนหัว (identified below) และบันทึกลงใน main file เป็นแบบ little endian ส่วน integers and double-precision floating point numbers จะแทนส่วนที่เหลือของ file และจะจัดการกับ file เป็นแบบ big endian

2.5.5.2 The Main File Header

ส่วนหัวของ main file จะมีความยาว 100 byte ในตารางที่ 2.1 จะแสดง fields ใน file header ร่วมกับตำแหน่งใน byte ต่าง ๆ ค่า (value) , ชนิด (type) และ byte order ในตารางนี้จะแสดงตำแหน่งความสัมพันธ์เริ่มต้นของ file

Position	Field	Value	Type	Byte Order
Byte 0	File Code	9994	Integer	Big
Byte 4	Unused	0	Integer	Big
Byte 8	Unused	0	Integer	Big
Byte 12	Unused	0	Integer	Big
Byte 16	Unused	0	Integer	Big
Byte 20	Unused	0	Integer	Big
Byte 24	File Length	File Length	Integer	Big
Byte 28	Version	1000	Integer	Little
Byte 32	Shape Type	Shape Type	Integer	Little
Byte 36	Bounding Box	Xmin	Double	Little
Byte 44	Bounding Box	Ymin	Double	Little
Byte 52	Bounding Box	Xmax	Double	Little
Byte 60	Bounding Box	Ymax	Double	Little
Byte 68*	Bounding Box	Zmin	Double	Little
Byte 76*	Bounding Box	Zmax	Double	Little
Byte 84*	Bounding Box	Mmin	Double	Little
Byte 92*	Bounding Box	Mmax	Double	Little

* Unused, with value 0.0, if not Measured or Z type

ตารางที่ 2.1 แสดงรายละเอียดของ main file header

ค่าสำหรับความยาวของ file เป็นผลรวมของความยาว file เป็น 16-bit words ทุกตำแหน่งที่ไม่เป็น null ใน Shapefile เป็นการร้องขอในขนาดของชนิดรูปร่างที่เหมือนกัน ค่าสำหรับชนิดรูปร่างจะเป็นดังต่อไปนี้

Value	Shape Type
0	Null Shape
1	Point
3	PolyLine
5	Polygon
8	MultiPoint
11	PointZ
13	PolyLineZ
15	PolygonZ
18	MultiPointZ
21	PointM
23	PolyLineM
25	PolygonM
28	MultiPointM
31	MultiPatch

ตารางที่ 2.2 แสดงค่าประจำตัวต่าง ๆ ใน Shape Type

Shape type ที่นอกเหนือจากนี้ (2, 4, 6, ect, และมากกว่า 33) เป็นตัวที่สำรองไว้ใช้ในอนาคต ซึ่งในปัจจุบัน Shape file ถูกจำกัดให้เหมือนกับชนิดของ shape นอกเหนือจากตัวพิเศษ ในอนาคต shapefile อาจจะประกอบด้วยมากกว่าหนึ่งชนิด shape ถ้ารวมชนิดของ shape เป็นการดำเนินการ field shape type ในส่วนหัวต้องการตัวบ่งชี้ file ที่เป็นอะไรก็ตาม

Bounding Box ในส่วนหัวของ main file จะเก็บขนาดที่เป็นอยู่จริงของ shape ใน file ซึ่ง Bounding ที่น้อยที่สุดของรูปสี่เหลี่ยมมุมฉากเส้นตั้งฉากจะเป็นแกน X และ Y (และที่เป็นไปได้เป็น M และ Z) ทุกแกนที่ประกอบเป็นรูปร่าง ถ้า shapefile ว่างเปล่า (ไม่มี record) ค่าสำหรับ Xmin, Ymin, Xmax และ Ymax คือไม่ได้ระบุไว้ ส่วน Mmin และ Mmax สามารถจะประกอบไปด้วย ค่าของ “no data” สำหรับ shapefile ของการควบคุมโดย shape type นั้นจะไม่ถูกควบคุม

2.5.5.3 Record Header

สำหรับส่วนหัวแต่ละ record จะเก็บตัวเลขและ ขนาดความยาวสำหรับ record ส่วนหัวของ record จะมีการกำหนดขนาดเป็น 8 byte ในตารางที่ 2.3 จะแสดงความสัมพันธ์ของ file header กับตำแหน่ง byte, value, type และ byte order ในตารางจะแสดงตำแหน่งความสัมพันธ์ของ record เริ่มต้น

Position	Field	Value	Type	Byte Order
Byte 0	Record Number	Record Number	Integer	Big
Byte 4	Content Length	Content Length	Integer	Big

ตารางที่ 2.3 แสดงรายละเอียดของ Main File Record Headers

ตัวเลขในการ record จะเริ่มต้นที่เลข 1 ขนาดความยาว record มีขนาดเป็นความยาวของขนาด record ที่ตัดออกเป็น 16-bit words ซึ่งแต่ละ record ดังนั้นให้เป็น (4+ขนาดความยาว) 16-bit words ไปยังผลรวมขนาดของ file ที่เก็บที่ 24 byte ในส่วนหัว

2.5.5.4 Main File Record Content

Shapefile จะเก็บขนาดที่ประกอบไปด้วย shape type ตามด้วยข้อมูล geometric สำหรับรูปร่าง ขนาดของความจุ record ขึ้นอยู่กับตัวเลขของเส้นแบ่ง และจุดตัดของเส้นใน shape สำหรับแต่ละชนิดของรูปร่างเราจะอธิบายรูปร่างในส่วนที่เป็น mapping ส่วนประกอบ record บน disk ในตารางที่ 2.4 จาก 16 ตำแหน่งร่วมกับความสัมพันธ์ของส่วนประกอบ record

2.5.5.5 Null Shapes

ชนิดของรูปร่างของ 0 แสดงเป็น null shape คือไม่มีข้อมูล geometric สำหรับรูปร่าง แต่ละลักษณะชนิด (point , line , polygon , ect) จะรองรับ nulls มันเป็นการพบว่ามีจุดและ ไม่มีจุดที่เหมือนกับ shapefile ถ้ามี null shape บ่อย ๆ เราจะทำการสร้าง shapefile และร่วมกับข้อมูล geometry ก่อนหลังจากที่เราสร้าง

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	0	Integer	1	Little

ตารางที่ 2.4 แสดงความจุ Null Shape Record

2.5.5.6 Shape Type in X , Y Space

Point จะประกอบไปด้วยคู่ของ double-precision พิกัดใน X , Y

Point

{

Double X //X coordinate


```

        Double Y      //Y coordinate
    }

```

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	1	Integer	1	Little
Byte 4	X	X	Double	1	Little
Byte 12	Y	Y	Double	1	Little

ตารางที่ 2.5 Point Record Contents

Multipoint multipoint จะถูกแทนด้วย set ของจุดดังข้างล่างนี้

```

MultiPoint
{
    Double[4] Box // Bounding Box
    Integer NumPoints // Number of Points
    Point[NumPoints] Points // The Points in the Set
}

```

Bounding Box จะเก็บ Xmin , Ymin , Xmax , Ymax

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	8	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumPoints	NumPoints	Integer	1	Little
Byte 40	Points	Points	Point	NumPoints	Little

ตารางที่ 2.6 Multipoint Record Contents

PolyLine polyline เป็นชุดของจุดตัดของเส้นที่ประกอบไปด้วย หนึ่งหรือมากกว่า หนึ่งเส้น เส้นเป็นการติดต่อลำดับของจุดสองจุด หรือมากกว่านั้น เพราะรายละเอียดไม่อนุญาตให้เป็นจุดที่เรียงต่อเนื่องกัน ร่วมกันเหมือนเป็นพิกัด ตัวอ่าน shapefile

```

PolyLine
{
    Double[4] Box // Bounding Box
    Integer NumParts // Number of Parts
    Integer NumPoints // Total Number of Points
}

```

Integer[NumParts] Parts // Index to First Point in Part

Point[NumPoints] Points // Points for All Parts

}

สำหรับแต่ละ field ของ polyline จะอธิบายรายละเอียดดังนี้

Box Bounding Box สำหรับ PolyLine จะเก็บค่าของ Xmin, Ymin, Xmax, Ymax

NumParts เป็นตัวเลขของเส้นใน PolyLine

Numpoints เป็นตัวเลขผลรวมของจุดสำหรับทุก ๆ เส้น

Parts เป็น array ของความยาว NumParts จะเก็บสำหรับแต่ละ PolyLine โดย index ของจุดแรกในจุด array ส่วน array indexes รวมกับความสัมพันธ์ ศูนย์

Point เป็น array ของความยาว NumPoint จุดสำหรับแต่ละเส้นใน PolyLine เป็นการเก็บแบบ end to end จุดสำหรับ Part 2 ตามด้วยจุดสำหรับ Part 1 และที่เหลือ

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	3	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumParts	NumParts	Integer	1	Little
Byte 40	NumPoints	NumPoints	Integer	1	Little
Byte 44	Parts	Parts	Integer	NumParts	Little
Byte X	Points	Points	Point	NumPoints	Little

Note: $X = 44 + 4 * \text{NumParts}$

ตารางที่ 2.7 PolyLine Record Contents

Polygon polygon จะประกอบไปด้วย หนึ่งหรือมากกว่าหนึ่งวงแหวน วงแหวนเป็นลำดับที่ต่อเนื่องกันไปเรื่อย ๆ สิ่งจุดหรือมากกว่าสิ่งจุดในรูปแบบปิด Polygon อาจจะประกอบด้วยวงแหวนด้านนอกหลายวง ส่วนจุดตัดหรือการกำหนดทิศทาง สำหรับวงแหวนแสดงส่วนไหนของวงแหวนเป็นภายในของ polygon บริเวณใกล้เคียง อาจจะมีหลาย polygon ได้

โครงสร้างของ polygon เป็นเหมือนกันทุกประการกับโครงสร้างของ PolyLine ดังข้างล่างนี้

Polygon

```

{
    Double[4] Box // Bounding Box
    Integer NumParts // Number of Parts
    Integer NumPoints // Total Number of Points
    Integer[NumParts] Parts // Index to First Point in Part
    Point[NumPoints] Points // Points for All Parts
}

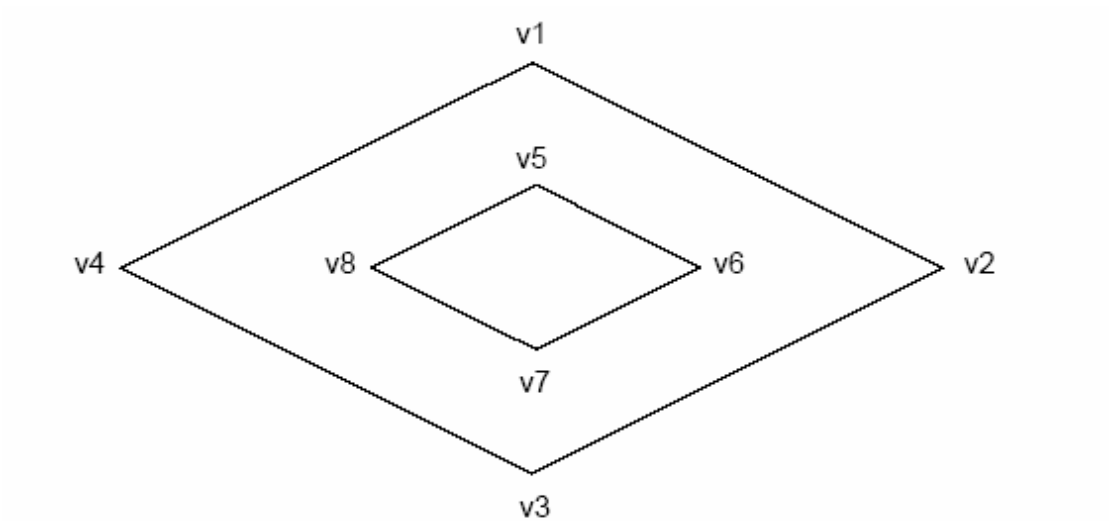
```

Box	Bounding Box สำหรับ Polygon จะเก็บค่าของ Xmin, Ymin, Xmax, Ymax
NumParts	เป็นตัวเลขของวงแหวนใน Polygon
Numpoints	เป็นตัวเลขผลรวมของจุดสำหรับทุก ๆ วงแหวน
Parts	เป็น array ของความยาว NumParts จะเก็บสำหรับแต่ละวงแหวน โดย index ของจุดแรกในจุด array ส่วน array indexes รวมกับความสัมพันธ์ ศูนย์
Point	เป็น array ของความยาว NumPoint จุดสำหรับแต่ละวงแหวนใน Polygon เป็นการเก็บแบบ end to end วงแหวนสำหรับ Ring 2 ตาม ด้วยวงแหวนสำหรับ Ring 1 และที่เหลือ

ในรูปที่ 2.9 จะแสดงให้เห็นถึงแนวคิดกว้าง ๆ การแทนที่ของ Polygon รูปที่แสดงนี้ polygon กับหนึ่งช่อง และผลรวมของจุดตัดทั้ง 8

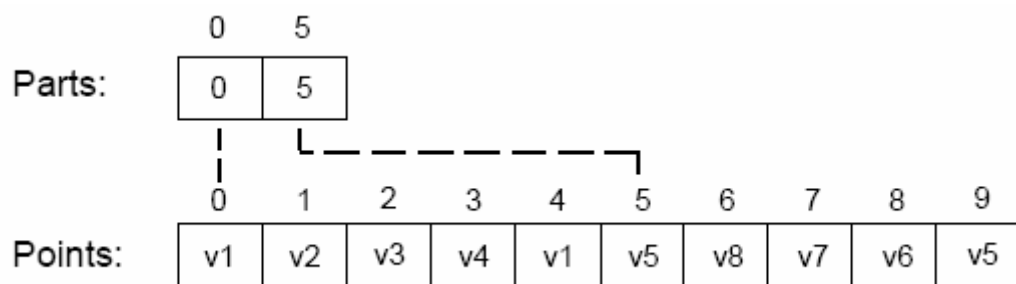
ข้างล่างนี้เป็นส่วนสำคัญที่เก็บรูปร่างของ Polygon

- วงแหวนปิด
 - วงแหวนในจุด array ไม่เป็นที่น่าสนใจ
 - Polygon จะเก็บใน shapefile เป็นสิ่งที่สำคัญอย่างดี
1. ไม่มีจุดตัดของตัวเอง หมายถึงอาจจะมีเพียงหนึ่งวงหรือ อาจจะซ้อนทับกับหลายวงก็ได้ วงแหวน polygon สามารถสัมผัสแต่ละจุดตัดได้ แต่ไม่เป็นไปในทิศทางเดียวกัน
 2. Polygon มีภายในที่ถูกต้องเป็นเส้นที่กำหนดขึ้นมา



รูปที่ 2.9 แสดงตัวอย่างของรูป Polygon

สำหรับตัวอย่างนี้ NumParts เท่ากับ 2 และ NumPoint เท่ากับ 10 ข้อสังเกตของจุดสำหรับรูปร่างของ Polygon เป็นดังต่อไปนี้



รูปที่ 2.10 แสดงการเก็บของ Parts และ Points

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	5	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumParts	NumParts	Integer	1	Little
Byte 40	NumPoints	NumPoints	Integer	1	Little
Byte 44	Parts	Parts	Integer	NumParts	Little
Byte X	Points	Points	Point	NumPoints	Little

Note: $X = 44 + 4 * \text{NumParts}$

ตารางที่ 2.8 Polygon Record Contents

2.5.5.7 Measured Shape Type in X,Y Space

ชนิดของรูปร่างนี้มีการเพิ่มฟิลด์ M ข้อสังเกตที่ “no data” ค่าของมันสามารถระบุค่าของ M (ให้ดูที่ Numeric Type)

PointM pointM จะประกอบไปด้วยคู่ของฟิลด์ double-precision ใน X , Y บวกการวัด M

```
PointM
{
    Double X // X coordinate
    Double Y // Y coordinate
    Double M // Measure
}
```

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	21	Integer	1	Little
Byte 4	X	X	Double	1	Little
Byte 12	Y	Y	Double	1	Little
Byte 20	M	M	Double	1	Little

ตารางที่ 2.9 PointM Record Contents

MultipointM multipointM แทนด้วยเซตของ PointM ดังนี้

```
MultiPointM
{
    Double[4] Box // Bounding Box
    Integer NumPoints // Number of Points
    Point[NumPoints] Points // The Points in the Set
    Double[2] M Range // Bounding Measure Range
    Double[NumPoints] M Array // Measures
}
```

สำหรับ field ของ multipointM จะเป็นดังนี้

Box Bounding Box สำหรับ MultipointM จะเก็บค่าของ Xmin, Ymin, Xmax, Ymax

NumParts เป็นตัวเลขของ Points

Numpoints เป็น array ของ point ของความยาว NumPoint

M Range	ปริมาณที่น้อยสุดและมากที่สุด สำหรับ MultipointM ซึ่งจะเก็บใน Mmin , Mmax
M Array	เป็น array ของปริมาณของความยาว NumPoints

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	28	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumPoints	NumPoints	Integer	1	Little
Byte 40	Points	Points	Point	NumPoints	Little
Byte X*	Mmin	Mmin	Double	1	Little
Byte X+8*	Mmax	Mmax	Double	1	Little
Byte X+16*	Marray	Marray	Double	NumPoints	Little
Note: X = 40 + (16 * NumPoints)					
* optional					

ตารางที่ 2.10 MultiPointM Record Contents

PolyLineM shapefile ของ PolyLineM จะประกอบไปด้วยเส้นหนึ่งหรือมากกว่าหนึ่งเส้น ซึ่งเส้นนั้นจะเรียงติดต่อกันของจุดสองจุดหรือมากกว่า

PolyLineM

```
{
    Double[4] Box // Bounding Box
    Integer NumParts // Number of Parts
    Integer NumPoints // Total Number of Points
    Integer[NumParts] Parts // Index to First Point in Part
    Point[NumPoints] Points // Points for All Parts
    Double[2] M Range // Bounding Measure Range
    Double[NumPoints] M Array // Measures for All Points
}
```

สำหรับ field ของ PolyLineM จะเป็นดังนี้

Box	Bounding Box สำหรับ PolyLineM จะเก็บค่าของ Xmin, Ymin, Xmax,Ymax
NumParts	เป็นตัวเลขของเส้นใน PolyLineM
Numpoints	เป็นผลรวมของ point สำหรับทุกเส้น

Parts	เป็น array ของความยาว NumParts จะเก็บสำหรับแต่ละเส้น โดย index ของจุดแรกใน array ส่วน array indexes รวมกับความสัมพัทธ์เป็นศูนย์
Point	เป็น array ของความยาว NumPoint จุดสำหรับแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Points สำหรับ parts 2 ตามด้วย point สำหรับ part 1 และที่เหลือ
M Range	ปริมาณที่น้อยสุดและมากที่สุด สำหรับ PolyLineM ซึ่งจะเก็บใน Mmin , Mmax
M Array	เป็น array ของความยาว NumPoints ปริมาณของแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Parts สำหรับ parts 2 ตามด้วย point สำหรับ part 1 และที่เหลือ

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	23	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumParts	NumParts	Integer	1	Little
Byte 40	NumPoints	NumPoints	Integer	1	Little
Byte 44	Parts	Parts	Integer	NumParts	Little
Byte X	Points	Points	Point	NumPoints	Little
Byte Y*	Mmin	Mmin	Double	1	Little
Byte Y + 8*	Mmax	Mmax	Double	1	Little
Byte Y + 16*	Marray	Marray	Double	NumPoints	Little

Note: $X = 44 + (4 * \text{NumParts})$, $Y = X + (16 * \text{NumPoints})$
 * optional

ตารางที่ 2.11 PolyLineM Record Contents

PolygonM PolygonM จะประกอบไปด้วยตัวเลขของวงแหวน เป็นรูปแบบวงแหวนปิด ข้อสังเกตการคำนวณในพื้นที่ X, Y ไม่ใช่ในพื้นที่ X, Y, M ซึ่ง PolygonM จะประกอบไปด้วยวงแหวนภายนอกหลาย ๆ วง ซึ่งวงแหวนของ PolygonM เป็นการอ้างอิงถึงเส้นของมันเอง

โครงสร้างของ PolygonM เป็นเหมือนกันทุกประการกับโครงสร้าง PolyLineM

PolygonM

{

Double[4] Box // Bounding Box

Integer NumParts // Number of Parts

Integer NumPoints // Total Number of Points

```

Integer[NumParts] Parts // Index to First Point in Part
Point[NumPoints] Points // Points for All Parts
Double[2] M Range // Bounding Measure Range
Double[NumPoints] M Array // Measures for All Points
}

```

สำหรับ field ของ PolygonM จะเป็นดังนี้

Box Bounding Box สำหรับ PolygonM จะเก็บค่าของ Xmin, Ymin, Xmax, Ymax

NumParts เป็นตัวเลขของเส้นใน PolygonM

Numpoints เป็นผลรวมของ point สำหรับทุกวงแหวน

Parts เป็น array ของความยาว NumParts จะเก็บสำหรับแต่ละเส้น โดย index ของจุดแรกใน array ส่วน array indexes ร่วมกับความสัมพันธ์เป็นศูนย์

Point เป็น array ของความยาว NumPoint จุดสำหรับแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Points สำหรับ Ring 2 ตามด้วย point สำหรับ Ring 1 และที่เหลือ

M Range ปริมาณที่น้อยสุดและมากที่สุด สำหรับ PolyLineM ซึ่งจะเก็บใน Mmin , Mmax

M Array เป็น array ของความยาว NumPoints ปริมาณของแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Parts สำหรับปริมาณ Ring 2 ตามด้วยปริมาณสำหรับ Ring 1 และที่เหลือ

ข้างล่างนี้เป็นส่วนสำคัญที่เก็บรูปร่างของ Polygon

- วงแหวนปิด
- วงแหวนในจุด array ไม่เป็นที่น่าสนใจ

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	25	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumParts	NumParts	Integer	1	Little
Byte 40	NumPoints	NumPoints	Integer	1	Little
Byte 44	Parts	Parts	Integer	NumParts	Little
Byte X	Points	Points	Point	NumPoints	Little
Byte Y*	Mmin	Mmin	Double	1	Little
Byte Y + 8*	Mmax	Mmax	Double	1	Little
Byte Y + 16*	Marray	Marray	Double	NumPoints	Little

Note: X = 44 + (4 * NumParts), Y = X + (16 * NumPoints)
 * optional

ตารางที่ 2.12 PolygonM Record Contents

2.5.5.8 Shape Types in X,Y,Z Space รูปร่างของชนิดนี้มีชนิดเป็นพิกัดที่เลือกได้ M ข้อสังเกตคือ “no data” ค่าของมันสามารถระบุค่าได้ สำหรับ M (ดูในเรื่อง Numeric Type)

PointZ pointZ จะประกอบไปด้วย 3 ตัวของพิกัด double-precision ใน X, Y, Z บวกปริมาณ

```
PointZ
{
    Double X // X coordinate
    Double Y // Y coordinate
    Double Z // Z coordinate
    Double M // Measure
}
```

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	11	Integer	1	Little
Byte 4	X	X	Double	1	Little
Byte 12	Y	Y	Double	1	Little
Byte 20	Z	Z	Double	1	Little
Byte 28	Measure	M	Double	1	Little

ตารางที่ 2.13 PointZ Record Contents

MultipointZ ถูกแทนด้วยเซตของ PointZ ดังข้างล่างนี้

```
MultiPointZ
{
    Double[4] Box // Bounding Box
    Integer NumPoints // Number of Points
    Point[NumPoints] Points // The Points in the Set
    Double[2] Z Range // Bounding Z Range
    Double[NumPoints] Z Array // Z Values
    Double[2] M Range // Bounding Measure Range
    Double[NumPoints] M Array // Measures
}
```

- Bounding Box สำหรับ PolygonM จะเก็บค่าของ Xmin, Ymin, Xmax, Ymax

- Bounding Z Range จะเก็บค่าของ Zmin , Zmax ส่วน Bounding M Range จะเก็บค่าของ Mmin , Mmax

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	18	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumPoints	NumPoints	Integer	1	Little
Byte 40	Points	Points	Point	NumPoints	Little
Byte X	Zmin	Zmin	Double	1	Little
Byte X+8	Zmax	Zmax	Double	1	Little
Byte X+16	Zarray	Zarray	Double	NumPoints	Little
Byte Y*	Mmin	Mmin	Double	1	Little
Byte Y+8*	Mmax	Mmax	Double	1	Little
Byte Y+16*	Marray	Marray	Double	NumPoints	Little
Note: X = 40 + (16 * NumPoints); Y = X + 16 + (8 * NumPoints)					
* optional					

ตารางที่ 2.14 MultiPointZ Record Contents

PolyLineZ PolyLineZ จะประกอบไปด้วยเส้นหนึ่งหรือมากกว่าหนึ่งเส้น ซึ่งเส้นนั้นจะเรียงติดต่อกันของจุดสองจุดหรือมากกว่าสองจุด

PolyLineZ

```
{
    Double[4] Box // Bounding Box
    Integer NumParts // Number of Parts
    Integer NumPoints // Total Number of Points
    Integer[NumParts] Parts // Index to First Point in Part
    Point[NumPoints] Points // Points for All Parts
    Double[2] Z Range // Bounding Z Range
    Double[NumPoints] Z Array // Z Values for All Points
    Double[2] M Range // Bounding Measure Range
    Double[NumPoints] M Array // Measures
}
```

สำหรับ fields ของ PolyLineZ จะอธิบายรายละเอียดดังข้างล่างนี้

Box Bounding Box สำหรับ PolygonM จะเก็บค่าของ Xmin, Ymin, Xmax, Ymax

NumParts เป็นตัวเลขของเส้นใน PolyLineZ

Numpoints	เป็นผลรวมของ point สำหรับทุกเส้น
Parts	เป็น array ของความยาว NumParts จะเก็บสำหรับแต่ละเส้น โดย index ของจุดแรกใน array ส่วน array indexes รวมกับความสัมพันธ์เป็นศูนย์
Point	เป็น array ของความยาว NumPoint จุดสำหรับแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Points สำหรับ Part 2 ตามด้วย point สำหรับ Part 1 และที่เหลือ
Z Range	ค่า Z ที่น้อยสุดและมากที่สุด สำหรับ PolyLineZ ซึ่งจะเก็บใน Zmin , Zmax
Z Array	เป็น array ของความยาว NumPoints ปริมาณของแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Parts ค่า Z สำหรับ Part 2 ตามด้วยค่า Z สำหรับ Part 1 และที่เหลือ
M Range	ปริมาณที่น้อยสุดและมากที่สุด สำหรับ PolyLineZ ซึ่งจะเก็บใน Mmin , Mmax
M Array	เป็น array ของความยาว NumPoints ปริมาณของแต่ละเส้นใน PolyLineZ เป็นการเก็บแบบ end to end ปริมาณสำหรับ Part 2 ตามด้วยปริมาณสำหรับ Part 1 และที่เหลือ

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	13	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumParts	NumParts	Integer	1	Little
Byte 40	NumPoints	NumPoints	Integer	1	Little
Byte 44	Parts	Parts	Integer	NumParts	Little
Byte X	Points	Points	Point	NumPoints	Little
Byte Y	Zmin	Zmin	Double	1	Little
Byte Y + 8	Zmax	Zmax	Double	1	Little
Byte Y + 16	Zarray	Zarray	Double	NumPoints	Little
Byte Z*	Mmin	Mmin	Double	1	Little
Byte Z+8*	Mmax	Mmax	Double	1	Little
Byte Z+16*	Marray	Marray	Double	NumPoints	Little
Note: $X = 44 + (4 * \text{NumParts})$, $Y = X + (16 * \text{NumPoints})$, $Z = Y + 16 + (8 * \text{NumPoints})$ * optional					

ตารางที่ 2.15 PolyLineZ Record Contents

PolygonZ PolygonZ จะประกอบไปด้วยตัวเลขของวงแหวน เป็นรูปแบบวงแหวนปิด ซึ่ง PolygonZ จะประกอบไปด้วยวงแหวนภายนอกหลาย ๆ วง ซึ่งวงแหวนของ PolygonZ เป็นการอ้างอิงถึงเส้นของมันเอง

โครงสร้างของ PolygonZ เป็นเหมือนกันทุกประการกับโครงสร้าง PolyLineZ

```

PolygonZ
{
    Double[4] Box // Bounding Box
    Integer NumParts // Number of Parts
    Integer NumPoints // Total Number of Points
    Integer[NumParts] Parts // Index to First Point in Part
    Point[NumPoints] Points // Points for All Parts
    Double[2] Z Range // Bounding Z Range
    Double[NumPoints] Z Array // Z Values for All Points
    Double[2] M Range // Bounding Measure Range
    Double[NumPoints] M Array // Measures
}

```

สำหรับ field ของ PolygonZ จะเป็นดังนี้

Box	Bounding Box สำหรับ PolygonM จะเก็บค่าของ Xmin, Ymin, Xmax,Ymax
NumParts	เป็นตัวเลขของวงแหวนใน PolygonZ
Numpoints	เป็นผลรวมของ point สำหรับทุกวงแหวน
Parts	เป็น array ของความยาว NumParts จะเก็บสำหรับแต่ละเส้น โดย index ของจุดแรกใน array ส่วน array indexes ร่วมกับความสัมพันธ์เป็นศูนย์
Point	เป็น array ของความยาว NumPoint จุดสำหรับแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Points สำหรับ Ring 2 ตามด้วย point สำหรับ Ring 1 และที่เหลือ
Z Range	ค่า Z ที่น้อยสุดและมากที่สุด สำหรับเก็บใน Zmin , Zmax
Z Array	เป็น array ของความยาว NumPoints ค่าของ Z สำหรับแต่ละวงแหวนใน PolygonZ เป็นการเก็บแบบ end to end ค่า Z สำหรับ Ring 2 ตามด้วยค่า Z สำหรับ Ring 1 และที่เหลือ
M Range	ปริมาณที่น้อยสุดและมากที่สุด สำหรับ PolygonZ เก็บใน Mmin , Mmax
M Array	เป็น array ของความยาว NumPoints ปริมาณสำหรับแต่ละวงแหวนใน PolygonZ เป็นการเก็บแบบ end to end ปริมาณสำหรับ Ring 2 ตามด้วยค่า Z สำหรับ Ring 1 และที่เหลือ

ข้างล่างนี้เป็นส่วนสำคัญที่เก็บเกี่ยวกับรูปร่างของ PolygonZ

- วงแหวนปิด
- วงแหวนในจุด array ไม่เป็นที่น่าสนใจ

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	15	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumParts	NumParts	Integer	1	Little
Byte 40	NumPoints	NumPoints	Integer	1	Little
Byte 44	Parts	Parts	Integer	NumParts	Little
Byte X	Points	Points	Point	NumPoints	Little
Byte Y	Zmin	Zmin	Double	1	Little
Byte Y+8	Zmax	Zmax	Double	1	Little
Byte Y+16	Zarray	Zarray	Double	NumPoints	Little
Byte Z*	Mmin	Mmin	Double	1	Little
Byte Z+8*	Mmax	Mmax	Double	1	Little
Byte Z+16*	Marray	Marray	Double	NumPoints	Little

Note: $X = 44 + (4 * \text{NumParts})$, $Y = X + (16 * \text{NumPoints})$, $Z = Y + 16 + (8 * \text{NumPoints})$

* optional

ตารางที่ 2.16 PolygonZ Record Contents

MultiPatch MultiPatch จะประกอบไปด้วยตัวเลขของพื้นผิว ซึ่งพื้นผิวของ MultiPatch เป็นการอ้างอิงถึงเส้นของมัน และชนิดของเส้นที่ควบคุมของเส้นตัดของ MultiPatch เส้นของ MultiPatch สามารถแบ่งเป็นชนิดได้ดังนี้

1. Triangle Strip เป็นการเชื่อมโยงของสี่เหลี่ยมมุมฉาก ที่ทุก ๆ จุดตัดของเส้นที่สมบูรณ์เป็นสี่เหลี่ยมมุมฉากใหม่ ซึ่งสี่เหลี่ยมมุมฉากใหม่เป็นรูปแบบของการติดต่อกันของจุดตัดใหม่กับทั้งสองที่กะทันหันแทนตำแหน่งเดิม

2. Triangle Fan เป็นการเชื่อมโยงของสี่เหลี่ยมรูปพัด ที่ทุก ๆ จุดตัดของเส้นที่สมบูรณ์เป็นสี่เหลี่ยมใหม่ ซึ่งสี่เหลี่ยมใหม่เป็นรูปแบบของการติดต่อกันของจุดตัดใหม่กับทั้งสองที่กะทันหันแทนตำแหน่งเดิม และจุดตัดแรกของเส้น

3. Outer Ring ด้านนอกวงแหวนของ Polygon

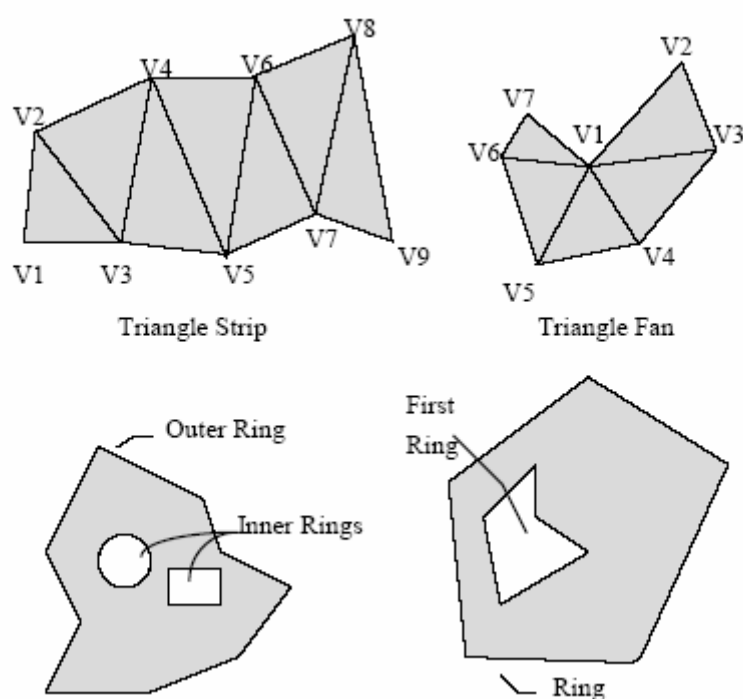
4. Inner Ring ภายในวงแหวนของ Polygon

5. First Ring วงแหวนวงแรกของ Polygon ของชนิดที่ไม่ได้ระบุไว้

6. Ring วงแหวนของ Polygon ของชนิดที่ไม่ได้ระบุไว้

สี่เหลี่ยมมุมฉากรูปเดียว หรือสี่เหลี่ยมรูปพัดจะถูกแทนด้วยพื้นผิวหนึ่งพื้นผิวดังรูปที่ 2.11 เป็นตัวอย่างของชนิดของเส้น

ลำดับของเส้นที่เป็นวงแหวนสามารถอธิบายเป็นพื้นผิวแบบ Polygon ได้โดยจะประกอบไปด้วยชนิดของ Outer Ring แทนด้วยเขตแดนด้านนอกของ patch ตามด้วยตัวเลขของ Inner Ring แทนด้วยช่องเพื่อให้เห็นความแตกต่างของชนิดวงแหวนเราจะแทนที่ด้วย พื้นที่หลายเหลี่ยม กับช่อง ลำดับแรกคือ First Ring ตามด้วยตัวเลขของ Ring ลำดับของ Ring จะไม่สำคัญกว่า First Ring จะรักษาลำดับของ Outer Ring ไว้ด้านนอกช่อง



รูปที่ 2.11 แสดงตัวอย่างของชนิดของเส้น MultiPatch

ค่าที่ใช้สำหรับ encoding ชนิดของเส้นเป็นดังนี้

Value	Part Type
0	Triangle Strip
1	Triangle Fan
2	Outer Ring
3	Inner Ring
4	First Ring
5	Ring

รูปที่ 2.12 แสดงค่าที่ใช้สำหรับการ encoding

MultiPatch

```
{  
  
    Double[4] Box // Bounding Box  
  
    Integer NumParts // Number of Parts  
  
    Integer NumPoints // Total Number of Points  
  
    Integer[NumParts] Parts // Index to First Point in Part  
  
    Integer[NumParts] PartTypes // Part Type  
  
    Point[NumPoints] Points // Points for All Parts  
  
    Double[2] Z Range // Bounding Z Range  
  
    Double[NumPoints] Z Array // Z Values for All Points  
  
    Double[2] M Range // Bounding Measure Range  
  
    Double[NumPoints] M Array // Measures  
  
}
```

สำหรับแต่ละ field ของ MultiPatch เป็น

Box	Bounding Box สำหรับ PolygonM จะเก็บค่าของ Xmin, Ymin, Xmax,Ymax
NumParts	เป็นตัวเลขของเส้นใน MultiPatch
Numpoints	เป็นผลรวมของ point สำหรับทุกเส้น
Parts	เป็น array ของความยาว NumParts จะเก็บสำหรับแต่ละเส้น โดย index ของจุดแรกใน array ส่วน array indexes รวมกับความสัมพันธ์เป็นศูนย์
Point	เป็น array ของความยาว NumPoint จุดสำหรับแต่ละเส้นใน PolyLineM เป็นการเก็บแบบ end to end Points สำหรับ Ring 2 ตามด้วย point สำหรับ Ring 1 และที่เหลือ
Z Range	ค่า Z ที่น้อยสุดและมากที่สุด สำหรับเก็บใน Zmin , Zmax
Z Array	เป็น array ของความยาว NumPoints ค่า Z ของแต่ละเส้นใน MultiPatch เป็นการเก็บแบบ end to end ค่า Z สำหรับ Part 2 ตามด้วยค่า Z สำหรับ Part 1 และที่เหลือ
M Range	ปริมาณที่น้อยสุดและมากที่สุด สำหรับ MultiPatch จะเก็บใน Mmin , Mmax
M Array	เป็น array ของความยาว NumPoints ปริมาณของแต่ละเส้นใน MultiPatch เป็นการเก็บแบบ end to end ปริมาณสำหรับ Part 2 ตามด้วยปริมาณสำหรับ Part 1 และที่เหลือ

ข้างล่างนี้เป็นส่วนสำคัญที่เก็บเกี่ยวกับรูปร่างของ MultiPatch

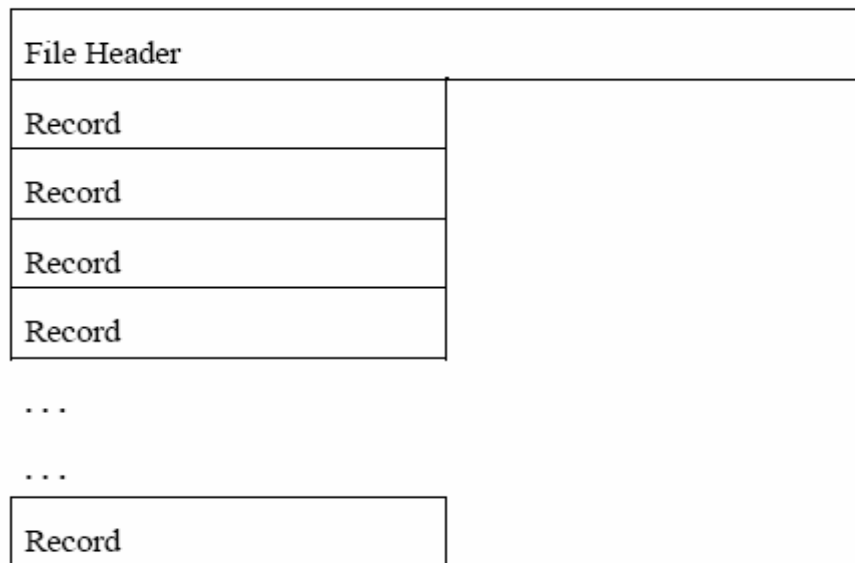
- ถ้าเส้นเป็นวงแหวนจะเป็นวงแหวนปิด
- ลำดับของเส้นที่เป็นวงแหวนใน point array มีความหมายเป็น Inner Ring ตามด้วยพวก Outer Ring ลำดับของ Ring แทนด้วยพื้นผิวหนึ่งส่วนร่วมกับ วงแหวนของชนิด First Ring
- เส้นต่าง ๆ สามารถเป็นเส้นแบ่งเขตร่วมได้ แต่เส้นเหล่านี้จะไม่ตัดขวางกัน และไม่เหลื่อมล้ำซึ่งกันและกัน

Position	Field	Value	Type	Number	Byte Order
Byte 0	Shape Type	31	Integer	1	Little
Byte 4	Box	Box	Double	4	Little
Byte 36	NumParts	NumParts	Integer	1	Little
Byte 40	NumPoints	NumPoints	Integer	1	Little
Byte 44	Parts	Parts	Integer	NumParts	Little
Byte W	PartTypes	PartTypes	Integer	NumParts	Little
Byte X	Points	Points	Point	NumPoints	Little
Byte Y	Zmin	Zmin	Double	1	Little
Byte Y+8	Zmax	Zmax	Double	1	Little
Byte Y+16	Zarray	Zarray	Double	NumPoints	Little
Byte Z*	Mmin	Mmin	Double	1	Little
Byte Z+8*	Mmax	Mmax	Double	1	Little
Byte Z+16*	Marray	Marray	Double	NumPoints	Little
Note: $W = 44 + (4 * \text{NumParts})$, $X = W + (4 * \text{NumParts})$, $Y = X + (16 * \text{NumPoints})$, $Z = Y + 16 + (8 * \text{NumPoints})$ * optional					

ตารางที่ 2.17 MultiPatch Record Contents

2.5.7 องค์ประกอบของ Index File

Index file (.shx) จะประกอบไปด้วยส่วนหัวที่มีขนาด 100byte ตามด้วย 8 byte จำกัดความยาวของ record ดังรูปที่ 2.13 จะแสดงถึงส่วนประกอบต่าง ๆ ของ index file



รูปที่ 2.13 ส่วนประกอบของ Index File

2.5.7.1 The Index File Header

ส่วนหัวของ Index file มีส่วนประกอบเหมือนกับ header ของ Main file ยิ่งไปกว่านั้น ขนาดของการเก็บ file ใน Index file header มีผลรวมขนาดของ Index file เป็น 16-bit words

2.5.7.2 Index Records

ในหนึ่ง record ใน Index file จะเก็บ Offset และขนาดของ Content แต่สำหรับหนึ่ง record ของ main file ในตารางที่ 2.18 จะแสดงรายละเอียดของ field ใน file header

Position	Field	Value	Type	Byte Order
Byte 0	Offset	Offset	Integer	Big
Byte 4	Content Length	Content Length	Integer	Big

ตารางที่ 2.18 รายละเอียดของ Index Records

Offset ของ record ใน main file เป็นตัวเลขของ 16-bit words จาก byte เริ่มต้นของ main file ถึง byte แรกของ record header ดังนั้น offset สำหรับ record แรกใน main file เป็น 50 ให้ header มี 100 byte

ความยาว Content จะเก็บใน index record เป็นเหมือนกับค่าที่เก็บใน main file record header

2.5.8 องค์ประกอบของ dBASE File

dBASE file (.dbf) จะประกอบไปด้วยจุดเด่นของ attribute หรือ key ของ attribute ซึ่งสามารถ joined กับตารางอื่นได้ มันเป็นรูปแบบมาตรฐาน DBF file ที่ใช้มากในโปรแกรมประยุกต์ของ Windows™ และ DOS. ชุดของ field สามารถแทนในตารางมีสามสิ่งที่จะต้องจำเป็นอย่างนี้

1. ชื่อของ file เป็นเหมือนส่วนที่เติมข้างหน้า shape และ index file ส่วนที่ต้องทำจะเป็น .bdf (สามารถดูในตัวอย่างเรื่อง Naming Conventions)
2. ในตารางจะประกอบไปด้วยหนึ่ง record ต่อ ลักษณะของรูปร่าง
3. ใน record จะเหมือนกับลักษณะของรูปร่างใน main file (.shp)
4. ค่าของ year ใน dBASE header จะเริ่มตั้งแต่ 1900